



Universidade Federal do Rio de Janeiro

www.paraosebaixo.com.br

Enzo Vieira, Caio David, Luís Rafael

Agosto, 2026

1	Matematica	1	5	Arvore	19
2	Geometria	7	6	DataStructures	21
3	Strings	11	7	DP	24
4	Grafos	13	8	Extra	25

1 Matematica

1.1 Identidades

Series

$$\sum_{i=1}^n i^2 = \frac{n(n+1)(2n+1)}{6} \quad \sum_{i=1}^n i^3 = \frac{n^2(n+1)^2}{4} = \left(\sum_{i=1}^n i\right)^2$$

$$g_k(n) = \sum_{i=1}^n i^k = \frac{1}{k+1} \left(n^{k+1} + \sum_{j=1}^k \binom{k+1}{j+1} (-1)^{j+1} g_{k-j}(n) \right)$$

$$\sum_{i=0}^n ic^i = \frac{nc^{n+2} - (n+1)c^{n+1} + c}{(c-1)^2}, \quad c \neq 1$$

$$\sum_{i=0}^{\infty} ic^i = \frac{c}{(1-c)^2}, \quad |c| < 1$$

Binomial Identities

- $\binom{n}{k} = \frac{n}{k} \binom{n-1}{k-1} = \binom{n-1}{k-1} + \binom{n-1}{k}$
- $\binom{n}{h} \binom{n-h}{k} = \binom{n}{k} \binom{n-k}{h}$
- $\sum k \binom{n}{k} = n2^{n-1}$
- $\sum_{j=0}^k \binom{m}{j} \binom{n-m}{k-j} = \binom{n}{k}$
- $\sum_{m=k}^n \binom{m}{k} = \binom{n+1}{k+1}$
- $(x+y)^n = \sum \binom{n}{k} x^{n-k} y^k$

$$\sum k^2 \binom{n}{k} = (n+n^2)2^{n-2}$$

$$\sum \binom{m}{j}^2 = \binom{2m}{m}$$

$$\sum_{k=0}^{\lfloor n/2 \rfloor} \binom{n-k}{k} = F_{n+1}$$

$$(1+x)^n = \sum \binom{n}{k} x^k$$

Progressão Aritmética (PA)

Termo Geral: $a_n = a_1 + (n-1)r$
Soma dos Termos: $S_n = \frac{(a_1+a_n)n}{2}$
PA de 2ª Ordem: $a_n = An^2 + Bn + C$

Progressão Geométrica (PG)

Termo Geral: $a_n = a_1 \cdot q^{n-1}$
Soma Finita: $S_n = a_1 \frac{q^n - 1}{q - 1}$
Soma Infinita ($|q| < 1$): $S_{\infty} = \frac{a_1}{1-q}$

Funções Geradoras

$$(1+x)^n = \sum_{k \geq 0} \binom{n}{k} x^k$$

$$\frac{1}{(1-x)^{m+1}} = \sum_{k \geq 0} \binom{k+m}{m} x^k$$

$$\frac{x^m}{(1-x)^{m+1}} = \sum_{k \geq 0} \binom{k}{m} x^k$$

1.2 Number Theory

Identities

$$\sum_{d|n} \varphi(d) = n$$

$$\sum_{\substack{i < n \\ \gcd(i,n)=1}} i = n \cdot \frac{\varphi(n)}{2}$$

$$|\{(x,y) : 1 \leq x,y \leq n, \gcd(x,y) = 1\}| = \sum_{d=1}^n \mu(d) \left\lfloor \frac{n}{d} \right\rfloor^2$$

$$\sum_{x=1}^n \sum_{y=1}^n \gcd(x,y) = \sum_{k=1}^n k \sum_{k|l} \left\lfloor \frac{n}{l} \right\rfloor^2 \mu\left(\frac{l}{k}\right)$$

$$\sum_{x=1}^n \sum_{y=x}^n \gcd(x,y) = \sum_{g=1}^n \sum_{g|d} d \cdot \varphi\left(\frac{d}{g}\right)$$

$$\sum_{x=1}^n \sum_{y=1}^n \text{lcm}(x,y) = \sum_{d=1}^n d \mu(d) \sum_{d|l} l \left(\left\lfloor \frac{n}{l} \right\rfloor + 1 \right)^2$$

$$\sum_{x=1}^n \sum_{y=x+1}^n \text{lcm}(x,y) = \sum_{g=1}^n \sum_{g|d} d \cdot \varphi\left(\frac{d}{g}\right) \cdot \frac{d}{g} \cdot \frac{1}{2}$$

$$\sum_{x \in A} \sum_{y \in A} \gcd(x,y) = \sum_{t=1}^n \left(\sum_{l|t} \frac{t}{l} \mu(l) \right) \left(\sum_{t|a} \text{freq}[a] \right)^2$$

$$\sum_{x \in A} \sum_{y \in A} \text{lcm}(x,y) = \sum_{t=1}^n \left(\sum_{l|t} \frac{l}{t} \mu(l) \right) \left(\sum_{a \in A, t|a} a \right)^2$$

Large Prime Gaps

For numbers until 10^9 the largest gap is 400.
 For numbers until 10^{18} the largest gap is 1500.

Prime counting function - $\pi(x)$

The prime counting function is asymptotic to $\frac{x}{\log x}$, by the prime number theorem.

x	10	10 ²	10 ³	10 ⁴	10 ⁵	10 ⁶	10 ⁷
$\pi(x)$	4	25	168	1 229	9 592	78 498	664 579

Some Primes

999999937 1000000007 1000000009 1000000021
 1000000033 $10^{18} - 11$ $10^{18} + 3$ 2305843009213693951 = $2^{61} - 1$

Number of Divisors

n	6	60	360	5040	55440	720720	4324320
d(n)	4	12	24	60	120	240	384
n	367567200	6983776800	13967553600	321253732800			
d(n)	1152	2304	2688	5376			

$18401055938125660800 \approx 2e18$ is highly composite with 184320 divisors.

For numbers up to 10^{88} , $d(n) < 3.6 \sqrt[3]{n}$.

Lucas's Theorem

$$\binom{n}{m} \equiv \prod_{i=0}^k \binom{n_i}{m_i} \pmod{p}$$

For p prime. n_i and m_i are the coefficients of the representations of n and m in base p . In particular, $\binom{n}{m}$ is odd if and only if n is a submask of m .

Fermat's Theorems

Let p be a prime number and a an integer, then:

$$a^p \equiv a \pmod{p}$$

$$a^{p-1} \equiv 1 \pmod{p}$$

Lemma: Let p be a prime number and a and b integers, then:

$$(a+b)^p \equiv a^p + b^p \pmod{p}$$

Lemma: Let p be a prime number and a an integer. The inverse of a modulo p is a^{p-2} :

$$a^{-1} \equiv a^{p-2} \pmod{p}$$

Taking modulo at the exponent

If a and m are coprime, then

$$a^n \equiv a^{n \bmod \varphi(m)} \pmod{m}$$

Generally, if $n \geq \log_2 m$, then

$$a^n \equiv a^{\varphi(m) + [n \bmod \varphi(m)]} \pmod{m}$$

Mobius inversion

If $g(n) = \sum_{d|n} f(d)$, then $f(n) = \sum_{d|n} g(d) \mu(\frac{n}{d})$.

A more useful definition is: $\sum_{d|n} \mu(d) = [n=1]$

Example, sum of LCM:

$$\begin{aligned} \sum_{i=1}^n \sum_{j=1}^n \text{lcm}(i, j) &= \sum_{k=1}^n \sum_{i=1}^n \sum_{j=1}^n [\gcd(i, j) = k] \frac{ij}{k} \\ &= \sum_{k=1}^n \sum_{a=1}^{\lfloor \frac{n}{k} \rfloor} \sum_{b=1}^{\lfloor \frac{n}{k} \rfloor} [\gcd(a, b) = 1] abk \\ &= \sum_{k=1}^n k \sum_{a=1}^{\lfloor \frac{n}{k} \rfloor} a \sum_{b=1}^{\lfloor \frac{n}{k} \rfloor} b [d|a] [d|b] \mu(d) \\ &= \sum_{k=1}^n k \sum_{d=1}^{\lfloor \frac{n}{k} \rfloor} \mu(d) \left(\sum_{a=1}^{\lfloor \frac{n}{kd} \rfloor} [d|a] a \right) \left(\sum_{b=1}^{\lfloor \frac{n}{kd} \rfloor} [d|b] b \right) \\ &= \sum_{k=1}^n k \sum_{d=1}^{\lfloor \frac{n}{k} \rfloor} \mu(d) \left(\sum_{p=1}^{\lfloor \frac{n}{kd} \rfloor} p \right) \left(\sum_{q=1}^{\lfloor \frac{n}{kd} \rfloor} q \right) \end{aligned}$$

Chicken McNugget Theorem

Given two **coprime** numbers n, m , the largest number that cannot be written as a linear combination of them is $nm - n - m$.

- There are $\frac{(n-1)(m-1)}{2}$ non-negative integers that cannot be written as a linear combination of n and m ;
- For each pair $(k, nm - n - m - k)$, for $k \geq 0$, exactly one can be written.

Harmonic Lemma

This technique computes sums of the form $\sum_{i=1}^n f(\lfloor \frac{n}{i} \rfloor)$ in $O(\sqrt{n})$. The value of $\lfloor \frac{n}{i} \rfloor$ is constant over blocks. If we are at index l , the value $v = \lfloor \frac{n}{l} \rfloor$ remains constant up to index $r = \lfloor \frac{n}{v} \rfloor$. We can iterate through these blocks instead of individual indices.

```
long long sum = 0;
for (int l = 1, r; l <= n; l = r + 1) {
    int val = n / l;
    r = n / val;
    sum += (long long)(r - l + 1) * f(val);
}
```

Generalization: For sums like $\sum_{i=1}^n g(i) \cdot f(\lfloor \frac{n}{i} \rfloor)$, the contribution of a block $[l, r]$ is:

$$(G(r) - G(l-1)) \cdot f\left(\left\lfloor \frac{n}{l} \right\rfloor\right)$$

where $G(k)$ is the prefix sum $\sum_{i=1}^k g(i)$, which must be efficiently computable.

Goldbach's Conjecture

Every even integer greater than 2 is the sum of two prime numbers. Verified for all even numbers up to 4×10^{18}

1.3 Combinatória

Stars and Bars Bounded

De quantas formas podemos definir uma sequencia x tal que:

$$\begin{aligned} \sum_{i=0}^n x_i &= M \\ l_i \leq x_i &\leq r_i \end{aligned}$$

Existência de solução A existência de solução é garantida se e somente se:

$$\sum_{i=0}^n l_i \leq M \leq \sum_{i=0}^n r_i$$

Solução com DP em $O(NM^2)$

$$DP(N, M) = \sum_{i=l_n}^{r_n} DP(N-1, M-i)$$

$$DP(0, M) = \begin{cases} 0, & M \neq 0 \\ 1, & M = 0 \end{cases}$$

Caso Particular $l_i = 0, r_i = M$ Stars and Bars com N-1 Barras

$$Resposta = \binom{M+N-1}{M}$$

Redução do problema para $l_i = 0$

$$M \leftarrow M - \sum_{i=0}^n l_i$$

$$x_i \leftarrow x_i - l_i$$

$$r_i \leftarrow r_i - l_i$$

$$l_i \leftarrow 0$$

Caso Especial $r_i = R$

$$Resposta = \sum_{k=0}^N (-1)^k \binom{N}{k} \binom{M - (R+1)k + N - 1}{N-1}$$

Formulas Básicas

Permutação Circular

O número de maneiras de ordenar n objetos distintos em um círculo.

$$PC_n = (n-1)!$$

Permutação com Repetição

O número de maneiras de ordenar n objetos, onde existem n_1 objetos idênticos de um tipo, n_2 de outro, e assim por diante.

$$P_n^{n_1, n_2, \dots, n_k} = \frac{n!}{n_1! n_2! \dots n_k!}$$

Combinação Caótica (Desarranjo)

O número de permutações de n objetos onde nenhum objeto permanece em sua posição original.

$$D_n = n! \sum_{k=0}^n \frac{(-1)^k}{k!}$$

Aproximação para n grande:

$$D_n \approx \frac{n!}{e}$$

Generalization for calculating number of elements in exactly r sets

A princípio da inclusão-exclusão pode ser reescrita para calcular o número de elementos que estão presentes em zero conjuntos:

$$\left| \bigcap_{i=1}^n A_i \right| = \sum_{m=0}^n (-1)^m \sum_{|X|=m} \left| \bigcap_{i \in X} A_i \right|$$

Considere sua generalização para calcular o número de elementos que estão presentes em exatamente r conjuntos:

$$\left| \bigcup_{|B|=r} \left[\bigcap_{i \in B} A_i \cap \bigcap_{j \notin B} A_j \right] \right| = \sum_{m=r}^n (-1)^{m-r} \binom{m}{r} \sum_{|X|=m} \left| \bigcap_{i \in X} A_i \right|$$

- **Catalan:** $C_n = \frac{1}{n+1} \binom{2n}{n}$. 1, 1, 2, 5, 14, 42.
- **Stirling 2:** $S(n, k) = kS(n-1, k) + S(n-1, k-1)$.
- **Vandermonde:** $\sum_{k=0}^r \binom{n}{k} \binom{m}{r-k} = \binom{n+m}{r}$.
- **Hockey-Stick:** $\sum_{i=r}^n \binom{i}{r} = \binom{n+1}{r+1}$.

crivo.h

Description: Crivo (Sieve) + Fatoração. Precomputa primos e menor divisor primo. `factorize(n)` retorna os fatores primos de n com suas frequências.

Time: $O(n \log \log n)$ para o crivo, $O(\log n)$ por fatoração

```
struct Sieve{
    int maxn;
    vector<int> is_prime, min_div;
    Sieve(int n){
        this->maxn = n;
        is_prime.assign(n+1, 1);
        min_div.resize(n+1);

        for(int i = 0; i <= n; i++){
            min_div[i] = i;
        }

        is_prime[0] = is_prime[1] = 0;
        for (int i = 2; i <= n; i++) {
            if (is_prime[i] && (long long)i * i <= n) {
                for (int j = i * i; j <= n; j += i){
                    if(is_prime[j]) min_div[j] = i;
                    is_prime[j] = false;
                }
            }
        }
    }
};
```

```
    }
}

vector<pair<int,int>> factorize(int n){
    assert(n <= maxn);
    vector<pair<int,int>> fact;
    while(n > 1){
        if(fact.empty() || fact.back().first !=
            min_div[n]){
            fact.push_back({min_div[n], 1});
        }else{
            fact.back().second += 1;
        }
        n /= min_div[n];
    }
    return fact;
}
};
```

CRT.h

Description: Teorema Chinês do Resto (Generalizado). Combina equações $x \equiv a \pmod{m}$. Os módulos m não precisam ser coprimos. Se não houver solução, retorna $a = -1$.

Time: $O(\log m)$ por equação

```
template<typename T> tuple<T, T, T> ext_gcd(T a, T b) {
    if (!a) return {b, 0, 1};
    auto [g, x, y] = ext_gcd(b%a, a);
    return {g, y - b/a*x, x};
}

template<typename T = ll> struct crt {
    T a, m;

    crt() : a(0), m(1) {}
    crt(T a_, T m_) : a(a_), m(m_) {}
    crt operator * (crt C) {
        auto [g, x, y] = ext_gcd(m, C.m);
        if ((a - C.a) % g) a = -1;
        if (a == -1 or C.a == -1) return crt(-1, 0);
        T lcm = m/g*C.m;
        T ans = a + (x*(C.a-a)/g % (C.m/g))*m;
        return crt((ans % lcm + lcm) % lcm, lcm);
    }
};
```

EuclidesEstendido.h

Description: Algoritmo de Euclides Estendido. Retorna $\gcd(a, b)$, e coeficientes x, y tais que $ax + by = \gcd(a, b)$.

Time: $O(\log(\min(a, b)))$

```
int egcd(int a, int b, int& x, int& y) {
    if (b == 0) {
        x = 1;
        y = 0;
        return a;
    }
    int x1, y1;
    int d = egcd(b, a % b, x1, y1);
    x = y1;
    y = x1 - y1 * (a / b);
    return d;
}
```

FFT.h

Description: FFT para convolução de polinômios. `fft(a)` computa $\hat{f}(k) = \sum_x a[x] \exp(2\pi i k x / N)$. N deve ser potência de 2. $\text{conv}(a, b) = c$, onde $c[x] = \sum_i a[i] b[x-i]$. $\text{convMod} < M >$ para convolução módulo M usando FFT de alta precisão. Arredondamento é seguro se $(\sum a_i^2 + \sum b_i^2) \log_2 N < 9 \cdot 10^{14}$.

Time: $O(N \log N)$ com $N = |A| + |B|$

```
struct FFT{
    typedef complex<double> C;
    typedef vector<double> vd;
    typedef vector<long long int> vl;
    typedef vector<int> vi;

    /*
     * Author: Ludo Pulles, chilli, Simon Lindholm
     * Date: 2019-01-09
     * License: CC0
     * Source:
     *   http://neerc.ifmo.ru/trains/toulouse/2017/fft2.pdf
     *   (do read, it's excellent)
     * Accuracy bound from
     *   http://www.daemonology.net/papers/fft.pdf
     * Description: fft(a) computes  $\hat{f}(k) = \sum_x a[x] \exp(2\pi i \cdot k \cdot x / N)$  for all  $k$ .  $N$ 
     *   must be a power of 2.
     * Useful for convolution:
     *   \texttt{conv(a, b) = c}, where  $c[x] = \sum a[i] b[x-i]$ .
     * For convolution of complex numbers or more than two
     *   vectors: FFT, multiply
     *   pointwise, divide by n, reverse(start+1, end), FFT
     *   back.
     * Rounding is safe if  $(\sum a_i^2 + \sum b_i^2) \log_2 N < 9 \cdot 10^{14}$ 
     *   (in practice  $10^{16}$ ); higher for random inputs).
     * Otherwise, use NTT/FFTMod.
     * Time:  $O(N \log N)$  with  $N = |A| + |B|$  ( $\tilde{O}$  if
     *   for  $N = 2^{22}$ )
     * Status: somewhat tested
     * Details: An in-depth examination of precision for
     *   both FFT and FFTMod can be found
     *   here (https://github.com/simonlindholm/fft-precision)
     *   n/blob/master/fft-precision.md)
     */
    void fft(vector<C>& a) {
        int n = a.size(), L = 31 - __builtin_clz(n);
        static vector<complex<long double>> R(2, 1);
        static vector<C> rt(2, 1); // (~ 10% faster if
        // double)
        for (static int k = 2; k < n; k *= 2) {
            R.resize(n); rt.resize(n);
            auto x = polar(1.0L, acos(-1.0L) / k);
            for(int i=k; i<2*k; i++) rt[i] = R[i] = i&1 ?
                x : R[i/2];
        }
        vi rev(n);
        for(int i = 0; i < n; i++) rev[i] = (rev[i / 2] | (i
            // & 1) << L) / 2;
        for(int i = 0; i < n; i++) if (i < rev[i])
            swap(a[i], a[rev[i]]);
        for (int k = 1; k < n; k *= 2)
            for (int i = 0; i < n; i += 2 * k) for(int j = 0;
                // j < k; j++) {
                // C z = rt[j+k] * a[i+j+k]; // (25% faster if
                //   hand-rolled) // include-line
                auto x = (double *) &rt[j+k], y = (double
                // *) &a[i+j+k]; // exclude-line
```

```

    C z(x[0]*y[0] - x[1]*y[1], x[0]*y[1] +
    ↪ x[1]*y[0]); // exclude-line
    a[i + j + k] = a[i + j] - z;
    a[i + j] += z;
}
}

vd conv(const vd& a, const vd& b) {
    if (a.empty() || b.empty()) return {};
    vd res(a.size() + b.size() - 1);
    int L = 32 - __builtin_clz(res.size()), n = 1 << L;
    vector<C> in(n), out(n);
    copy(a.begin(), a.end(), begin(in));
    for(int i = 0; i < b.size(); i++) in[i].imag(b[i]);
    fft(in);
    for (C& x : in) x *= x;
    for(int i = 0; i < n; i++) out[i] = in[-i & (n - 1)]
    ↪ - conj(in[i]);
    fft(out);
    for(int i = 0; i < res.size(); i++) res[i] =
    ↪ imag(out[i]) / (4 * n);
    return res;
}

vl conv(const vl& a, const vl& b) {
    if (a.empty() || b.empty()) return {};
    vd res(a.size() + b.size() - 1);
    int L = 32 - __builtin_clz(res.size()), n = 1 << L;
    vector<C> in(n), out(n);
    copy(a.begin(), a.end(), begin(in));
    for(int i = 0; i < b.size(); i++) in[i].imag(b[i]);
    fft(in);
    for (C& x : in) x *= x;
    for(int i = 0; i < n; i++) out[i] = in[-i & (n - 1)]
    ↪ - conj(in[i]);
    fft(out);
    for(int i = 0; i < res.size(); i++) res[i] =
    ↪ imag(out[i]) / (4 * n);
    vl r(a.size() + b.size() - 1);
    for(int i = 0; i < res.size(); i++) r[i] =
    ↪ (ll)(res[i]+.5);
    return r;
}

/*
 * Author: chilli
 * Date: 2019-04-25
 * License: CC0
 * Source:
    ↪ http://neerc.ifmo.ru/trains/toulouse/2017/fft2.pdf
 * Description: Higher precision FFT, can be used for
    ↪ convolutions modulo arbitrary integers
 * as long as  $N \log_2 N \cdot \text{mod} < 8.6 \cdot \text{mod}$ 
    ↪  $10^{14}$  (in practice  $10^{16}$  or higher).
 * Inputs must be in  $\mathbb{Z}_\text{mod}$ .
 * Time:  $O(N \log N)$ , where  $N = |A| + |B|$  (twice as
    ↪ slow as NTT or FFT)
 * Status: stress-tested
 * Details: An in-depth examination of precision for
    ↪ both FFT and FFTMod can be found
 * here (https://github.com/simonlindholm/fft-precision)
    ↪ n/blob/master/fft-precision.md)
 */
// multiplica dois polinomios modulo algum inteiro
template<int M> vl convMod(const vl& a, const vl& b) {
    if (a.empty() || b.empty()) return {};
    vl res(a.size() + b.size() - 1);
    int B=32-__builtin_clz(res.size()), n=1<<B,
    ↪ cut=int(sqrt(M));

```

```

vector<C> L(n), R(n), outs(n), outl(n);
for(int i = 0; i < a.size(); i++) L[i] = C((int)a[i]
    ↪ / cut, (int)a[i] % cut);
for(int i = 0; i < b.size(); i++) R[i] = C((int)b[i]
    ↪ / cut, (int)b[i] % cut);
fft(L), fft(R);
for(int i = 0; i < n; i++) {
    int j = -i & (n - 1);
    outl[j] = (L[i] + conj(L[j])) * R[i] / (2.0 * n);
    outs[j] = (L[i] - conj(L[j])) * R[i] / (2.0 * n)
    ↪ / 1i;
}
fft(outl), fft(outs);
for(int i = 0; i < res.size(); i++) {
    ll av = ll(real(outl[i])+.5), cv =
    ↪ ll(imag(outs[i])+.5);
    ll bv = ll(imag(outl[i])+.5) +
    ↪ ll(real(outs[i])+.5);
    res[i] = ((av % M * cut + bv) % M * cut + cv) % M;
}
return res;
}

};

```

flatDivisors.h

Description: Gera os divisores de todos os números até n e armazena em um vetor simples contíguo.

Usage: `build_flat_divisors()`; `for(int i = head[Y]; i < head[Y + 1]; ++i) int d = flat_divs[i];`

Time: $O(N \log N)$

```

int div_count[MAXN + 1]; // Stores how many divisors each
    ↪ number has
int head[MAXN + 2]; // Stores the starting index in the
    ↪ flat array for each number
int current_pos[MAXN + 1]; // Used to track where to insert
    ↪ the next divisor
vector<int> flat_divs; // The single flat array

```

```

void build_flat_divisors() {
    for(int i = 1; i <= MAXN; i++) {
        for(int j = i; j <= MAXN; j += i) {
            div_count[j]++;
        }
    }

    head[1] = 0;
    for(int i = 1; i <= MAXN; i++) {
        head[i + 1] = head[i] + div_count[i];
        current_pos[i] = head[i];
    }

    flat_divs.resize(head[MAXN + 1]);

    for(int i = 1; i <= MAXN; i++) {
        for(int j = i; j <= MAXN; j += i) {
            flat_divs[current_pos[j]++] = i;
        }
    }
}

```

FST.h

Description: Transform to a basis with fast convolutions of the form $c[z] = \sum_{z=x \oplus y} a[x] \cdot b[y]$, where $\oplus$ is one of AND, OR, XOR. The size of a must be a power of two.

Time: $O(N \log N)$

```

void FST(vector<int> &a, bool inv) {
    for (int n = a.size(), step = 1; step < n; step *= 2) {
        for (int i = 0; i < n; i += 2 * step) for(int j = i; j <
            ↪ i+step; ++j) {
            int &u = a[j], &v = a[j + step]; tie(u, v) =
                inv ? pii(v - u, u) : pii(v, u + v); // AND
            inv ? pii(v, u - v) : pii(u + v, u); // OR
            pii(u + v, u - v); // XOR
        }
    }
    if (inv) for(auto &x : a) x /= a.size(); // XOR only
}

vector<int> conv(vector<int> a, vector<int> b) {
    FST(a, 0); FST(b, 0);
    for(int i = 0; i < a.size(); ++i) a[i] *= b[i];
    FST(a, 1); return a;
}

```

gauss.h

Description: Eliminação de Gauss para resolver sistemas lineares. Retorna $(num_solucoes, solucao)$. 0 = sem solução, 1 = única, INF = infinitas.

Time: $O(n^3)$, onde n é o número de variáveis

```

template<typename T>
pair<int, vector<T>> gauss(vector<vector<T>> a, vector<T> b) {
    const double eps = 1e-6;
    int n = a.size(), m = a[0].size();
    for (int i = 0; i < n; i++) a[i].push_back(b[i]);
}

```

```

vector<int> where(m, -1);
for (int col = 0, row = 0; col < m and row < n; col++) {
    int sel = row;
    for (int i=row; i<n; ++i)
        if (abs(a[i][col]) > abs(a[sel][col])) sel = i;
    if (abs(a[sel][col]) < eps) continue;
    for (int i = col; i <= m; i++)
        swap(a[sel][i], a[row][i]);
    where[col] = row;

    for (int i = 0; i < n; i++) if (i != row) {
        T c = a[i][col] / a[row][col];
        for (int j = col; j <= m; j++)
            a[i][j] -= a[row][j] * c;
    }
    row++;
}

```

```

vector<T> ans(m, 0);
for (int i = 0; i < m; i++) if (where[i] != -1)
    ans[i] = a[where[i]][m] / a[where[i]][i];
for (int i = 0; i < n; i++) {
    T sum = 0;
    for (int j = 0; j < m; j++)
        sum += ans[j] * a[i][j];
    if (abs(sum - a[i][m]) > eps)
        return pair(0, vector<T>());
}

for (int i = 0; i < m; i++) if (where[i] == -1)
    return pair(INF, ans);
return pair(1, ans);
}

```

gcdConvolution.h

Description: Convolução de GCD. Computa $C[k] = \sum_{\gcd(i,j)=k} A[i] \cdot B[j]$. Usa transformadas Multiple Zeta/Möbius sobre primos.
Time: $O(n \log \log n)$

```
vector<int> PrimeEnumerate(int n) {
    vector<int> P; vector<bool> B(n + 1, 1);
    for (int i = 2; i <= n; i++) {
        if (B[i]) P.push_back(i);
        for (int j : P) { if (i * j > n) break; B[i * j] = 0; if
            ↪ (i % j == 0) break; }
        return P;
    }
    template<typename T>
    void MultipleZetaTransform(vector<T>& v) {
        const int n = (int)v.size() - 1;
        for (int p : PrimeEnumerate(n)) {
            for (int i = n / p; i; i--)
                v[i] += v[i * p];
        }
    }
    template<typename T>
    void MultipleMobiusTransform(vector<T>& v) {
        const int n = (int)v.size() - 1;
        for (int p : PrimeEnumerate(n)) {
            for (int i = 1; i * p <= n; i++)
                v[i] -= v[i * p];
        }
    }
    template<typename T>
    vector<T> GCDConvolution(vector<T> A, vector<T> B) {
        MultipleZetaTransform(A);
        MultipleZetaTransform(B);
        for (int i = 0; i < A.size(); i++) A[i] *= B[i];
        MultipleMobiusTransform(A);
        return A;
    }
}
```

Interpolation.h

Description: Interpolação de Lagrange. Dados $n + 1$ pontos, interpola um polinômio de grau $\leq n$ e avalia em um ponto x .
Time: $O(n^2)$

```
struct Interpolation
{
    //naive implementation  $O(n^2)$ 
    void interpolate(vector<pair<ll, ll>> &P, int x){

        ll ans = 0;
        for(int i = 0; i < P.size(); i++){
            ll li = 1;
            for(int j = 0; j < P.size(); j++){
                if(i == j) continue;
                li *= (x - P[j].first);
                li /= (P[i].first - P[j].first);
            }
            li *= P[i].second;
            ans += li;
        }
        return ans;
    }
};
```

Interpolation2.h

Description: Dados n pontos $(x[i], y[i])$, computa um polinômio de grau $n - 1$ que passa por eles: $p(x) = a[0]x^0 + \dots + a[n - 1]x^{n-1}$. Para precisão numérica, use $x[k] = c \cdot \cos(k/(n - 1) \cdot \pi)$, $k = 0 \dots n - 1$.
Time: $O(n^2)$

```
typedef vector<double> vd;
vd interpolate(vd x, vd y, int n) {
    vd res(n), temp(n);
    for(int k = 0; k < n - 1; ++k) for(int i = k + 1; i < n; ++i)
        y[i] = (y[i] - y[k]) / (x[i] - x[k]);
    double last = 0; temp[0] = 1;
    for(int k = 0; k < n; ++k) for(int i = 0; i < n; ++i) {
        res[i] += y[k] * temp[i]; swap(last, temp[i]);
        temp[i] -= last * x[k];
    }
    return res;
}
```

Matrix.h

Description: Multiplicação e exponenciação de matrizes. *operator** multiplica, *operator^* exponencia (exponenciação rápida).
Time: $O(n^3)$ para multiplicação, $O(n^3 \log e)$ para exponenciação

```
template<typename T> struct Matriz
{
    int n, m;
    vector<vector<T>> mat;

    static constexpr T add_id() { return 0; } //Neutral
    ↪ element of addition
    static constexpr T mul_id() { return 1; } //Neutral
    ↪ element of multiplication

    Matriz(int n, int m) : n(n), m(m), mat(n, vector<T> (m,
        ↪ add_id())) {}

    const T* operator[](int i) const { return mat[i].data(); }
    T* operator[](int i) { return mat[i].data(); }

    Matriz<T> operator*(const Matriz<T> &oth) const {
        assert(m == oth.n);
        Matriz<T> res(n, oth.m);
        for(int i = 0; i < n; i++) {
            for(int k = 0; k < m; k++) {
                T val = mat[i][k];
                for(int j = 0; j < oth.m; j++) {
                    // Add modulo here if the problem requires
                    ↪ it:
                    // res[i][j] = (res[i][j] + val * 1LL *
                    ↪ oth[k][j]) % MOD;
                    res[i][j] += val * oth[k][j];
                }
            }
        }
        return res;
    }

    Matriz<T> operator^(long long e) const {
        assert(n == m);
        Matriz<T> R(n, n), b = *this;
        for(int i = 0; i < n; i++) R[i][i] = mul_id();
        while (e > 0) {
            if (e & 1) R = R * b;
            b = b * b;
            e >>= 1;
        }
    }
}
```

```
    }
    return R;
};

using M = Matriz<int>;
```

ModInt.h

Description: Inteiro modular genérico com módulo primo p fixo em tempo de compilação. Suporta operações $+$, $-$, $*$, $/$, $\wedge$ e exponenciação rápida.
Usage: `typedef mod_int <(int)1e9+7> mint;`
Time: $O(\log N)$ por exponenciação/inversão

```
template<int p> struct mod_int {
    ll expo(ll b, ll e) {
        ll ret = 1;
        while (e) {
            if (e % 2) ret = ret * b % p;
            e /= 2, b = b * b % p;
        }
        return ret;
    }
    ll inv(ll b) { return expo(b, p-2); }

    using m = mod_int;
    int v;
    mod_int() : v(0) {}
    mod_int(ll v_) {
        if (v_ >= p or v_ <= -p) v_ %= p;
        if (v_ < 0) v_ += p;
        v = v_;
    }
    m& operator +=(const m& a) {
        v += a.v;
        if (v >= p) v -= p;
        return *this;
    }
    m& operator -=(const m& a) {
        v -= a.v;
        if (v < 0) v += p;
        return *this;
    }
    m& operator *=(const m& a) {
        v = v * ll(a.v) % p;
        return *this;
    }
    m& operator /=(const m& a) {
        v = v * inv(a.v) % p;
        return *this;
    }
    m operator -(){ return m(-v); }
    m& operator ^=(ll e) {
        if (e < 0) {
            v = inv(v);
            e = -e;
        }
        v = expo(v, e);
        // possível otimizacao:
        // cuidado com  $0^0$ 
        // v = expo(v, e%(p-1));
        return *this;
    }
    bool operator ==(const m& a) { return v == a.v; }
    bool operator !=(const m& a) { return v != a.v; }

    friend istream& operator >>(istream& in, m& a) {
        ll val; in >> val;
        a = m(val);
        return in;
    }
}
```

```

}
friend ostream& operator <<(ostream& out, m a) {
    return out << a.v;
}
friend m operator +(m a, m b) { return a += b; }
friend m operator -(m a, m b) { return a -= b; }
friend m operator *(m a, m b) { return a *= b; }
friend m operator /(m a, m b) { return a /= b; }
friend m operator ^(m a, ll e) { return a ^= e; }
};

typedef mod_int<(int)1e9+7> mint;

```

ModTemplate.h

Description: Template modular com pré-cálculo de fatoriais e inversos. *build()* pré-computa *fact[n]* e *ifact[n]* até *maxn*. *choose(a,b)* retorna $\binom{a}{b} \bmod \text{mod}$.
Time: $O(\text{maxn})$ para *build()*, $O(1)$ para *choose()*

```

const int mod = 1e9+7;

const int maxn = 1e6;
vector<ll> fact(maxn+1), ifact(maxn+1);

ll fastexp(ll b, ll e){
    ll res = 1;
    while(e){
        if(e&1) res = (res * b)%mod;
        b = (b * b)%mod;
        e/=2;
    }
    return res;
}

ll inv(ll x){
    return fastexp(x, mod-2);
}

ll choose(ll a, ll b){
    if(a < b) return 0;
    return fact[a] * ifact[b] %mod * ifact[a-b] %mod;
}

void build(){
    fact[0] = 1;
    for(int i = 1; i <= maxn; i++) fact[i] = (fact[i-1] *
        ↪ i)%mod;
    ifact[maxn] = inv(fact[maxn]);
    for(int i = maxn-1; i >= 0; i--) ifact[i] = (ifact[i+1] *
        ↪ (i+1))%mod;
}

```

NTT.h

```

/*
* Author: chilli
* Date: 2019-04-16
* License: CCO
* Source: based on KACTL's FFT
* Description: ntt(a) computes  $\hat{f}(k) = \sum_x a[x]$ 
    ↪  $g^{\{xk\}}$  for all  $k$ , where
    ↪  $g = \sqrt[n]{\text{root}^{\{(\text{mod}-1)/N\}}}$ .
* N must be a power of 2.

```

```

* Useful for convolution modulo specific nice primes of
    ↪ the form  $2^a b + 1$ ,
* where the convolution result has size at most  $2^a$ . For
    ↪ arbitrary modulo, see FFTMod.
\texttt{conv(a, b) = c}, where  $c[x] = \sum a[i]b[x-i]$ .
For manual convolution: NTT the inputs, multiply
pointwise, divide by n, reverse(start+1, end), NTT back.
* Inputs must be in  $[0, \text{mod})$ .
* Time:  $O(N \log N)$ 
* Status: stress-tested
*/

struct NTT
{
    typedef vector<long long int> vl;
    typedef vector<int> vi;

    const ll mod = (119 << 23) + 1, root = 62; // = 998244353
    // For  $p < 2^{30}$  there is also e.g.  $5 << 25$ ,  $7 << 26$ ,  $479$ 
    ↪  $<< 21$ 
    // and  $483 << 21$  (same root). The last two are  $> 10^9$ .
    void ntt(vl &a) {
        int n = a.size(), L = 31 - __builtin_clz(n);
        static vl rt(2, 1);
        for (static int k = 2, s = 2; k < n; k *= 2, s++) {
            rt.resize(n);
            ll z[] = {1, modpow(root, mod >> s)};
            for(int i = k; i < 2*k; i++) rt[i] = rt[i / 2] *
                ↪ z[i & 1] % mod;
        }
        vi rev(n);
        for(int i = 0; i < n; i++) rev[i] = (rev[i / 2] | (i
            ↪ & 1) << L) / 2;
        for(int i = 0; i < n; i++) if (i < rev[i]) swap(a[i],
            ↪ a[rev[i]]);
        for (int k = 1; k < n; k *= 2)
            for (int i = 0; i < n; i += 2 * k) for(int j = 0;
                ↪ j < k; j++) {
                ll z = rt[j + k] * a[i + j + k] % mod, &ai =
                    ↪ a[i + j];
                a[i + j + k] = ai - z + (z > ai ? mod : 0);
                ai += (ai + z >= mod ? z - mod : z);
            }
    }
    vl conv_ntt(const vl &a, const vl &b) {
        if (a.empty() || b.empty()) return {};
        int s = a.size() + b.size() - 1, B = 32 -
            ↪ __builtin_clz(s),
            n = 1 << B;
        int inv = modpow(n, mod - 2);
        vl L(a), R(b), out(n);
        L.resize(n), R.resize(n);
        ntt(L), ntt(R);
        for(int i = 0; i < n; i++)
            out[-i & (n - 1)] = (ll)L[i] * R[i] % mod * inv %
                ↪ mod;
        ntt(out);
        return {out.begin(), out.begin() + s};
    }
    ll modpow(ll b, ll e) {
        ll ans = 1;
        for (; e; b = b * b % mod, e /= 2)
            if (e & 1) ans = ans * b % mod;
        return ans;
    }
};

```

PollardHo.h

Description: Pollard's Rho para fatora  o de inteiros grandes. *prime(n)* testa primalidade com Miller-Rabin determin  stico. *fact(n)* retorna a lista de fatores primos (n   necessariamente   nicos).

Time: $O(n^{1/4})$ em m  dia, $O(n^{1/2})$ no pior caso

```

ll mul(ll a, ll b, ll m) {
    ll ret = a*b - ll((long double)1/m*a*b+0.5)*m;
    return ret < 0 ? ret+m : ret;
}

ll pow(ll x, ll y, ll m) {
    if (!y) return 1;
    ll ans = pow(mul(x, x, m), y/2, m);
    return y%2 ? mul(x, ans, m) : ans;
}

bool prime(ll n) {
    if (n < 2) return 0;
    if (n <= 3) return 1;
    if (n % 2 == 0) return 0;

    ll r = __builtin_ctzll(n - 1), d = n >> r;
    for (int a : {2, 325, 9375, 28178, 450775, 9780504,
        ↪ 1795265022}) {
        ll x = pow(a, d, n);
        if (x == 1 or x == n - 1 or a % n == 0) continue;

        for (int j = 0; j < r - 1; j++) {
            x = mul(x, x, n);
            if (x == n - 1) break;
        }
        if (x != n - 1) return 0;
    }
    return 1;
}

ll rho(ll n) {
    if (n == 1 or prime(n)) return n;
    auto f = [n](ll x) {return mul(x, x, n) + 1;};

    ll x = 0, y = 0, t = 30, prd = 2, x0 = 1, q;
    while (t % 40 != 0 or gcd(prd, n) == 1) {
        if (x==y) x = ++x0, y = f(x);
        q = mul(prd, abs(x-y), n);
        if (q != 0) prd = q;
        x = f(x), y = f(f(y)), t++;
    }
    return gcd(prd, n);
}

vector<ll> fact(ll n) {
    if (n == 1) return {};
    if (prime(n)) return {n};
    ll d = rho(n);
    vector<ll> l = fact(d), r = fact(n / d);
    l.insert(l.end(), r.begin(), r.end());
    return l;
}

```

segmentedSieve.h

Description: Prime sieve for generating all primes smaller than S.
Time: $S=1e9 \approx 1.5s$

```

const int LIM = 1e6;
bitset<LIM> isPrime;
vector<int> eratosthenes() {
    const int S = round(sqrt(LIM)), R = LIM/2;
    vector<int> pr = {2}, sieve(S+1);
    ⇨ pr.reserve(int(LIM/log(LIM)*1.1));
    vector<pair<int,int>> cp;
    for (int i = 3; i <= S; i += 2) if (!sieve[i]) {
        cp.push_back({i, i*i/2});
        for (int j = i*i; j <= S; j += 2*i) sieve[j] = 1;
    }
    for (int L = 1; L <= R; L += S) {
        array<bool, S> block{};
        for (auto &[p, idx] : cp)
            for (int i=idx; i < S+L; idx = (i+=p)) block[i-L] = 1;
        for (int i = 0; i < min(S, R - L); ++i)
            if (!block[i]) pr.push_back((L + i)*2 + 1);
    }
    for (int i : pr) isPrime[i] = 1;
    return pr;
}

```

2 Geometria

2.1 Pythagorean Triples

For all natural a, b, c satisfying $a^2 + b^2 = c^2$ there exist $m, n \in \mathbb{N}$ and $m > n$ such that (reverse is also true):

$$a = m^2 - n^2 \quad b = 2mn \quad c = m^2 + n^2$$

2.2 Heron's Formula

The area of a triangle can be written as $A = \sqrt{s(s-a)(s-b)(s-c)}$, where a, b, c are the lengths of its sides and $s = \frac{a+b+c}{2}$.

This can be generalized to compute the area A of a quadrilateral with sides a, b, c, d , with $s = \frac{a+b+c+d}{2}$ and α, γ any two opposite angles:

$$A = \sqrt{(s-a)(s-b)(s-c)(s-d) - abcd \left(\cos^2 \left(\frac{\alpha + \gamma}{2} \right) \right)}$$

2.3 Pick's Theorem

The area of a simple polygon whose vertices have integer coordinates is:

$$A = I + \frac{B}{2} - 1$$

where I is the number of interior integer points, and B is the number of integer points in the border of the polygon.

2.4 Colinear Points

Three points are colinear on $\mathbb{R}^2$ iff:

$$\begin{vmatrix} x_A & y_A & 1 \\ x_B & y_B & 1 \\ x_C & y_C & 1 \end{vmatrix} = 0$$

The absolute value of this determinant is twice the area of the triangle ABC .

2.5 Coplanar Points

Four points are coplanar in $\mathbb{R}^3$ iff:

$$\begin{vmatrix} x_A & y_A & z_A & 1 \\ x_B & y_B & z_B & 1 \\ x_C & y_C & z_C & 1 \\ x_D & y_D & z_D & 1 \end{vmatrix} = 0$$

2.6 Trigonometry

Angle Sum

$$\sin(a \pm b) = \sin a \cos b \pm \cos a \sin b$$

$$\cos(a \pm b) = \cos a \cos b \mp \sin a \sin b$$

$$\tan(a \pm b) = \frac{\tan a \pm \tan b}{1 \mp \tan a \tan b}$$

Sum-to-Product Transformation

$$\sin a \pm \sin b = 2 \sin \frac{a \pm b}{2} \cos \frac{a \mp b}{2}$$

$$\cos a + \cos b = 2 \cos \frac{a + b}{2} \cos \frac{a - b}{2}$$

$$\cos a - \cos b = -2 \sin \frac{a + b}{2} \sin \frac{a - b}{2}$$

$$\tan a \pm \tan b = \frac{\sin(a \pm b)}{\cos a \cos b}$$

2.7 Centroid of a polygon

The coordites of the centroid of a non-self-intersecting closed polygon is:

$$\frac{1}{3A} \left(\sum_{i=0}^{n-1} (x_i + x_{i+1})(x_i y_{i+1} - x_{i+1} y_i), \sum_{i=0}^{n-1} (y_i + y_{i+1})(x_i y_{i+1} - x_{i+1} y_i) \right)$$

where A is twice the signed area of the polygon.

2.8 Geometria Básica

Formas 2D

- **Triângulo:** $A = \frac{bh}{2} = \sqrt{s(s-a)(s-b)(s-c)}$, $s = \frac{a+b+c}{2}$
- **Círculo:** $C = 2\pi r = \pi d$ $A = \pi r^2$
- **Quadrado/Retângulo:** $A = s^2$ / $A = lw$ $P = 4s$ / $P = 2(l + w)$
- **Paralelogr./Trap.:** $A = bh$ $A = \frac{(a+b)h}{2}$

Formas 3D

- **Esfera:** $V = \frac{4}{3}\pi r^3$ $SA = 4\pi r^2$
- **Cilindro:** $V = \pi r^2 h$ $SA = 2\pi r(h + r)$, $L = 2\pi r h$
- **Cone:** $V = \frac{1}{3}\pi r^2 h$ $L = \pi r \sqrt{r^2 + h^2}$
- **Cubo/Prisma:** $V = s^3$ / $V = lwh$ $SA = 6s^2$ / $SA = 2(lw + lh + wh)$
- **Pirâmide:** $V = \frac{1}{3}A_b h$ (A_b : área da base)

Compare.h

Description: Ordena pontos por ângulo polar usando produto vetorial.

Time: $O(1)$ por comparação

```

long long cross(const point &a, const point &b) {
    return (long long) a.x * b.y - (long long) a.y * b.x;
}
bool cmp(const point &a, const point &b) {
    int ah = (a.y < 0 or (a.y == 0 and a.x < 0));
    int bh = (b.y < 0 or (b.y == 0 and b.x < 0));
    if (ah != bh) return ah > bh;
    return cross(a, b) < 0;
}

```

GeoDouble.h

Description: Geometria com double. Primitivas: ponto, reta, polígono, circunferência.

Time: $O(n \log n)$ convex hull, $O(n)$ demais

```

typedef double ld;
const ld DINF = 1e18;
const ld pi = acos(-1.0);
const ld eps = 1e-9;

#define sq(x) ((x)*(x))

bool eq(ld a, ld b) {
    return abs(a - b) <= eps;
}

struct pt { // ponto
    ld x, y;
    pt(ld x_ = 0, ld y_ = 0) : x(x_), y(y_) {}
    bool operator < (const pt p) const {
        if (!eq(x, p.x)) return x < p.x;
        if (!eq(y, p.y)) return y < p.y;
        return 0;
    }
    bool operator == (const pt p) const {
        return eq(x, p.x) and eq(y, p.y);
    }
    pt operator + (const pt p) const { return pt(x+p.x, y+p.y); }
    ⇨ }
    pt operator - (const pt p) const { return pt(x-p.x, y-p.y); }
    ⇨ }
    pt operator * (const ld c) const { return pt(x*c, y*c); }
    ⇨ }
    pt operator / (const ld c) const { return pt(x/c, y/c); }
    ⇨ }
    ld operator * (const pt p) const { return x*p.x + y*p.y; }
    ld operator ^ (const pt p) const { return x*p.y - y*p.x; }
    friend istream& operator >> (istream& in, pt& p) {
        return in >> p.x >> p.y;
    }
};

struct line { // reta
    pt p, q;
    line() {}
    line(pt p_, pt q_) : p(p_), q(q_) {}
}

```



```

friend istream& operator >> (istream& in, line& r) {
    return in >> r.p >> r.q;
}

// PONTO & VETOR

ld dist(pt p, pt q) { // distancia
    return hypot(p.y - q.y, p.x - q.x);
}

ld dist2(pt p, pt q) { // quadrado da distancia
    return sq(p.x - q.x) + sq(p.y - q.y);
}

ld norm(pt v) { // norma do vetor
    return dist(pt(0, 0), v);
}

ld angle(pt v) { // angulo do vetor com o eixo x
    ld ang = atan2(v.y, v.x);
    if (ang < 0) ang += 2*pi;
    return ang;
}

ld sarea(pt p, pt q, pt r) { // area com sinal
    return ((q-p)^(r-q))/2;
}

bool col(pt p, pt q, pt r) { // se p, q e r sao colin.
    return eq(sarea(p, q, r), 0);
}

bool ccw(pt p, pt q, pt r) { // se p, q, r sao ccw
    return sarea(p, q, r) > eps;
}

pt rotate(pt p, ld th) { // rotaciona o ponto th radianos
    return pt(p.x * cos(th) - p.y * sin(th),
             p.x * sin(th) + p.y * cos(th));
}

pt rotate90(pt p) { // rotaciona 90 graus
    return pt(-p.y, p.x);
}

// RETA

bool isvert(line r) { // se r eh vertical
    return eq(r.p.x, r.q.x);
}

bool isinseg(pt p, line r) { // se p pertence ao seg de r
    pt a = r.p - p, b = r.q - p;
    return eq((a ^ b), 0) and (a * b) < eps;
}

ld get_t(pt v, line r) { // retorna t tal que t*v pertence a
    // reta r
    return (r.p^r.q) / ((r.p-r.q)^v);
}

pt proj(pt p, line r) { // projecao do ponto p na reta r
    if (r.p == r.q) return r.p;
    r.q = r.q - r.p; p = p - r.p;
    pt proj = r.q * ((p*r.q) / (r.q*r.q));
    return proj + r.p;
}

pt inter(line r, line s) { // r inter s
    if (eq((r.p - r.q) ^ (s.p - s.q), 0)) return pt(DINF, DINF);

```

```

    r.q = r.q - r.p, s.p = s.p - r.p, s.q = s.q - r.p;
    return r.q * get_t(r.q, s) + r.p;
}

bool interseg(line r, line s) { // se o seg de r intersecta o
    // seg de s
    if (isinseg(r.p, s) or isinseg(r.q, s)
        or isinseg(s.p, r) or isinseg(s.q, r)) return 1;

    return ccw(r.p, r.q, s.p) != ccw(r.p, r.q, s.q) and
           ccw(s.p, s.q, r.p) != ccw(s.p, s.q, r.q);
}

ld disttoline(pt p, line r) { // distancia do ponto a reta
    return 2 * abs(sarea(p, r.p, r.q)) / dist(r.p, r.q);
}

ld disttoseg(pt p, line r) { // distancia do ponto ao seg
    if ((r.q - r.p)*(p - r.p) < 0) return dist(r.p, p);
    if ((r.p - r.q)*(p - r.q) < 0) return dist(r.q, p);
    return disttoline(p, r);
}

ld distseg(line a, line b) { // distancia entre seg
    if (interseg(a, b)) return 0;

    ld ret = DINF;
    ret = min(ret, disttoseg(a.p, b));
    ret = min(ret, disttoseg(a.q, b));
    ret = min(ret, disttoseg(b.p, a));
    ret = min(ret, disttoseg(b.q, a));

    return ret;
}

// POLIGONO

// corta poligono com a reta r deixando os pontos p tal que
// ccw(r.p, r.q, p)
vector<pt> cut_polygon(vector<pt> v, line r) { // 0(n)
    vector<pt> ret;
    for (int j = 0; j < v.size(); j++) {
        if (ccw(r.p, r.q, v[j])) ret.push_back(v[j]);
        if (v.size() == 1) continue;
        line s(v[j], v[(j+1)%v.size()]);
        pt p = inter(r, s);
        if (isinseg(p, s)) ret.push_back(p);
    }
    ret.erase(unique(ret.begin(), ret.end()), ret.end());
    if (ret.size() > 1 and ret.back() == ret[0]) ret.pop_back();
    return ret;
}

// distancia entre os retangulos a e b (lados paralelos aos
// eixos)
// assume que ta representado (inferior esquerdo, superior
// direito)
ld dist_rect(pair<pt, pt> a, pair<pt, pt> b) {
    ld hor = 0, vert = 0;
    if (a.second.x < b.first.x) hor = b.first.x - a.second.x;
    else if (b.second.x < a.first.x) hor = a.first.x -
        b.second.x;
    if (a.second.y < b.first.y) vert = b.first.y - a.second.y;
    else if (b.second.y < a.first.y) vert = a.first.y -
        b.second.y;
    return dist(pt(0, 0), pt(hor, vert));
}

ld polarea(vector<pt> v) { // area do poligono
    ld ret = 0;

```

```

    for (int i = 0; i < v.size(); i++)
        ret += sarea(pt(0, 0), v[i], v[(i + 1) % v.size()]);
    return abs(ret);
}

// se o ponto ta dentro do poligono: retorna 0 se ta fora,
// 1 se ta no interior e 2 se ta na borda
int inpol(vector<pt>& v, pt p) { // 0(n)
    int qt = 0;
    for (int i = 0; i < v.size(); i++) {
        if (p == v[i]) return 2;
        int j = (i+1)%v.size();
        if (eq(p.y, v[i].y) and eq(p.y, v[j].y)) {
            if ((v[i]-p)*(v[j]-p) < eps) return 2;
            continue;
        }
        bool baixo = v[i].y+eps < p.y;
        if (baixo == (v[j].y+eps < p.y)) continue;
        auto t = (p-v[i])^(v[j]-v[i]);
        if (eq(t, 0)) return 2;
        if (baixo == (t > eps)) qt += baixo ? 1 : -1;
    }
    return qt != 0;
}

bool interpol(vector<pt> v1, vector<pt> v2) { // se dois
    // poligonos se intersectam - 0(n*m)
    int n = v1.size(), m = v2.size();
    for (int i = 0; i < n; i++) if (inpol(v2, v1[i])) return 1;
    for (int i = 0; i < n; i++) if (inpol(v1, v2[i])) return 1;
    for (int i = 0; i < n; i++) for (int j = 0; j < m; j++)
        if (interseg(line(v1[i], v1[(i+1)%n]), line(v2[j],
            // v2[(j+1)%m]))) return 1;
    return 0;
}

ld distpol(vector<pt> v1, vector<pt> v2) { // distancia entre
    // poligonos
    if (interpol(v1, v2)) return 0;

    ld ret = DINF;

    for (int i = 0; i < v1.size(); i++) for (int j = 0; j <
        // v2.size(); j++)
        ret = min(ret, distseg(line(v1[i], v1[(i + 1) %
            // v1.size()]),
            line(v2[j], v2[(j + 1) % v2.size()])))
    return ret;
}

vector<pt> convex_hull(vector<pt> v) { // convex hull - 0(n
    // log(n))
    sort(v.begin(), v.end());
    v.erase(unique(v.begin(), v.end()), v.end());
    if (v.size() <= 1) return v;
    vector<pt> l, u;
    for (int i = 0; i < v.size(); i++) {
        while (l.size() > 1 and !ccw(l.end()[-2], l.end()[-1],
            // v[i]))
            l.pop_back();
        l.push_back(v[i]);
    }
    for (int i = v.size() - 1; i >= 0; i--) {
        while (u.size() > 1 and !ccw(u.end()[-2], u.end()[-1],
            // v[i]))
            u.pop_back();
        u.push_back(v[i]);
    }
    l.pop_back(); u.pop_back();

```

```

    for (pt i : u) l.push_back(i);
    return l;
}

struct convex_pol {
    vector<pt> pol;

    // nao pode ter ponto colinear no convex hull
    convex_pol() {}
    convex_pol(vector<pt> v) : pol(convex_hull(v)) {}

    // se o ponto ta dentro do hull - O(log(n))
    bool is_inside(pt p) {
        if (pol.size() == 0) return false;
        if (pol.size() == 1) return p == pol[0];
        int l = 1, r = pol.size();
        while (l < r) {
            int m = (l+r)/2;
            if (ccw(p, pol[0], pol[m])) l = m+1;
            else r = m;
        }
        if (l == 1) return isinseg(p, line(pol[0], pol[1]));
        if (l == pol.size()) return false;
        return !ccw(p, pol[l], pol[l-1]);
    }

    // ponto extremo em relacao a cmp(p, q) = p mais extremo q
    // (copiado de https://github.com/gustavoM32/caderno-zika)
    int extreme(const function<bool>(pt, pt)>& cmp) {
        int n = pol.size();
        auto extr = [&](int i, bool& cur_dir) {
            cur_dir = cmp(pol[(i+1)%n], pol[i]);
            return !cur_dir and !cmp(pol[(i+n-1)%n], pol[i]);
        };
        bool last_dir, cur_dir;
        if (extr(0, last_dir)) return 0;
        int l = 0, r = n;
        while (l+1 < r) {
            int m = (l+r)/2;
            if (extr(m, cur_dir)) return m;
            bool rel_dir = cmp(pol[m], pol[l]);
            if ((!last_dir and cur_dir) or
                (last_dir == cur_dir and rel_dir == cur_dir)) {
                l = m;
                last_dir = cur_dir;
            } else r = m;
        }
        return l;
    }

    int max_dot(pt v) {
        return extreme([&](pt p, pt q) { return p*v > q*v; });
    }

    pair<int, int> tangents(pt p) {
        auto L = [&](pt q, pt r) { return ccw(p, r, q); };
        auto R = [&](pt q, pt r) { return ccw(p, q, r); };
        return {extreme(L), extreme(R)};
    }
};

// CIRCUNFERENCIA

pt getcenter(pt a, pt b, pt c) { // centro da circunf dado 3
    pontos
    b = (a + b) / 2;
    c = (a + c) / 2;
    return inter(line(b, b + rotate90(a - b)),
                line(c, c + rotate90(a - c)));
}

vector<pt> circ_line_inter(pt a, pt b, pt c, ld r) { //
    intersecao da circunf (c, r) e reta ab

```

```

    vector<pt> ret;
    b = b-a, a = a-c;
    ld A = b*b;
    ld B = a*b;
    ld C = a*a - r*r;
    ld D = B*B - A*C;
    if (D < -eps) return ret;
    ret.push_back(c+a+b*(-B+sqrt(D+eps))/A);
    if (D > eps) ret.push_back(c+a+b*(-B-sqrt(D))/A);
    return ret;
}

vector<pt> circ_inter(pt a, pt b, ld r, ld R) { // intersecao
    da circunf (a, r) e (b, R)
    vector<pt> ret;
    ld d = dist(a, b);
    if (d > r+R or d+min(r, R) < max(r, R)) return ret;
    ld x = (d*d-R*R+r*r)/(2*d);
    ld y = sqrt(r*r-x*x);
    pt v = (b-a)/d;
    ret.push_back(a+v*x + rotate90(v)*y);
    if (y > 0) ret.push_back(a+v*x - rotate90(v)*y);
    return ret;
}

bool operator <(const line& a, const line& b) { // comparador
    pra reta
    // assume que as retas tem p < q
    pt v1 = a.q - a.p, v2 = b.q - b.p;
    if (!eq(angle(v1), angle(v2))) return angle(v1) < angle(v2);
    return ccw(a.p, a.q, b.p); // mesmo angulo
}

bool operator ==(const line& a, const line& b) {
    return !(a < b) and !(b < a);
}

// comparador pro set pra fazer sweep line com segmentos
struct cmp_sweepline {
    bool operator () (const line& a, const line& b) const {
        // assume que os segmentos tem p < q
        if (a.p == b.p) return ccw(a.p, a.q, b.q);
        if (!eq(a.p.x, a.q.x) and (eq(b.p.x, b.q.x) or a.p.x+eps
            < b.p.x))
            return ccw(a.p, a.q, b.p);
        return ccw(a.p, b.q, b.p);
    }
}

// comparador pro set pra fazer sweep angle com segmentos
pt dir;
struct cmp_sweepangle {
    bool operator () (const line& a, const line& b) const {
        return get_t(dir, a) + eps < get_t(dir, b);
    }
}

```

GeoInt.h

Description: Geometria com inteiros. Primitivas: ponto, reta, polígono.
Time: $O(n \log n)$ convex hull, $O(\log n)$ inside query, $O(n)$ demais

```
#define sq(x) ((x)*(x))
```

```

struct pt { // ponto
    int x, y;
    pt(int x_ = 0, int y_ = 0) : x(x_), y(y_) {}
    bool operator < (const pt p) const {
        if (x != p.x) return x < p.x;
        return y < p.y;
    }
}

```

```

}

bool operator ==(const pt p) const {
    return x == p.x and y == p.y;
}

pt operator + (const pt p) const { return pt(x+p.x, y+p.y); }
pt operator - (const pt p) const { return pt(x-p.x, y-p.y); }
pt operator * (const int c) const { return pt(x*c, y*c); }
ll operator * (const pt p) const { return x*(ll)p.x +
    y*(ll)p.y; }
ll operator ^ (const pt p) const { return x*(ll)p.y -
    y*(ll)p.x; }
friend istream& operator >> (istream& in, pt& p) {
    return in >> p.x >> p.y;
}

struct line { // reta
    pt p, q;
    line() {}
    line(pt p_, pt q_) : p(p_), q(q_) {}
    friend istream& operator >> (istream& in, line& r) {
        return in >> r.p >> r.q;
    }
};

// PONTO & VETOR

ll dist2(pt p, pt q) { // quadrado da distancia
    return sq(p.x - q.x) + sq(p.y - q.y);
}

ll sarea2(pt p, pt q, pt r) { // 2 * area com sinal
    return (q-p)^(r-q);
}

bool col(pt p, pt q, pt r) { // se p, q e r sao colin.
    return sarea2(p, q, r) == 0;
}

bool ccw(pt p, pt q, pt r) { // se p, q, r sao ccw
    return sarea2(p, q, r) > 0;
}

int quad(pt p) { // quadrante de um ponto
    return (p.x<0)^3*(p.y<0);
}

bool compare_angle(pt p, pt q) { // retorna se ang(p) < ang(q)
    if (quad(p) != quad(q)) return quad(p) < quad(q);
    return ccw(q, pt(0, 0), p);
}

pt rotate90(pt p) { // rotaciona 90 graus
    return pt(-p.y, p.x);
}

// RETA

bool isinseg(pt p, line r) { // se p pertence ao seg de r
    pt a = r.p - p, b = r.q - p;
    return (a ^ b) == 0 and (a * b) <= 0;
}

bool interseg(line r, line s) { // se o seg de r intersecta o
    seg de s
    if (isinseg(r.p, s) or isinseg(r.q, s)
        or isinseg(s.p, r) or isinseg(s.q, r)) return 1;
}

```

```

return ccw(r.p, r.q, s.p) != ccw(r.p, r.q, s.q) and
    ccw(s.p, s.q, r.p) != ccw(s.p, s.q, r.q);
}

int segpoints(line r) { // numero de pontos inteiros no
    ↪ segmento
    return 1 + __gcd(abs(r.p.x - r.q.x), abs(r.p.y - r.q.y));
}

double get_t(pt v, line r) { // retorna t tal que t*v pertence
    ↪ a reta r
    return (r.p^r.q) / (double) ((r.p-r.q)^v);
}

// POLIGONO

// quadrado da distancia entre os retangulos a e b (lados
    ↪ paralelos aos eixos)
// assume que ta representado (inferior esquerdo, superior
    ↪ direito)
ll dist2_rect(pair<pt, pt> a, pair<pt, pt> b) {
    int hor = 0, vert = 0;
    if (a.second.x < b.first.x) hor = b.first.x - a.second.x;
    else if (b.second.x < a.first.x) hor = a.first.x -
        ↪ b.second.x;
    if (a.second.y < b.first.y) vert = b.first.y - a.second.y;
    else if (b.second.y < a.first.y) vert = a.first.y -
        ↪ b.second.y;
    return sq(hor) + sq(vert);
}

ll polarea2(vector<pt> v) { // 2 * area do poligono
    ll ret = 0;
    for (int i = 0; i < v.size(); i++)
        ret += sarea2(pt(0, 0), v[i], v[(i + 1) % v.size()]);
    return abs(ret);
}

// se o ponto ta dentro do poligono: retorna 0 se ta fora,
// 1 se ta no interior e 2 se ta na borda
int inpol(vector<pt>& v, pt p) { // 0(n)
    int qt = 0;
    for (int i = 0; i < v.size(); i++) {
        if (p == v[i]) return 2;
        int j = (i+1)%v.size();
        if (p.y == v[i].y and p.y == v[j].y) {
            if ((v[i]-p)*(v[j]-p) <= 0) return 2;
            continue;
        }
        bool baixo = v[i].y < p.y;
        if (baixo == (v[j].y < p.y)) continue;
        auto t = (p-v[i])^(v[j]-v[i]);
        if (!t) return 2;
        if (baixo == (t > 0)) qt += baixo ? 1 : -1;
    }
    return qt != 0;
}

vector<pt> convex_hull(vector<pt> v) { // convex hull - 0(n
    ↪ log(n))
    sort(v.begin(), v.end());
    v.erase(unique(v.begin(), v.end()), v.end());
    if (v.size() <= 1) return v;
    vector<pt> l, u;
    for (int i = 0; i < v.size(); i++) {
        while (l.size() > 1 and !ccw(l.end()[-2], l.end()[-1],
            ↪ v[i]))
            l.pop_back();
        l.push_back(v[i]);
    }

```

```

for (int i = v.size() - 1; i >= 0; i--) {
    while (u.size() > 1 and !ccw(u.end()[-2], u.end()[-1],
        ↪ v[i]))
        u.pop_back();
    u.push_back(v[i]);
}
l.pop_back(); u.pop_back();
for (pt i : u) l.push_back(i);
return l;
}

ll interior_points(vector<pt> v) { // pontos inteiros dentro
    ↪ de um poligono simples
    ll b = 0;
    for (int i = 0; i < v.size(); i++)
        b += segpoints(line(v[i], v[(i+1)%v.size()]));
    return (polarea2(v) - b) / 2 + 1;
}

struct convex_pol {
    vector<pt> pol;

    // nao pode ter ponto colinear no convex hull
    convex_pol() {}
    convex_pol(vector<pt> v) : pol(convex_hull(v)) {}

    // se o ponto ta dentro do hull - 0(log(n))
    bool is_inside(pt p) {
        if (pol.size() == 0) return false;
        if (pol.size() == 1) return p == pol[0];
        int l = 1, r = pol.size();
        while (l < r) {
            int m = (l+r)/2;
            if (ccw(p, pol[0], pol[m])) l = m+1;
            else r = m;
        }
        if (l == 1) return isinseg(p, line(pol[0], pol[1]));
        if (l == pol.size()) return false;
        return !ccw(p, pol[l], pol[l-1]);
    }

    // ponto extremo em relacao a cmp(p, q) = p mais extremo q
    // (copiado de https://github.com/gustavoM32/caderno-zika)
    int extreme(const function<bool(pt, pt)>& cmp) {
        int n = pol.size();
        auto extr = [&](int i, bool& cur_dir) {
            cur_dir = cmp(pol[(i+1)%n], pol[i]);
            return !cur_dir and !cmp(pol[(i+n-1)%n], pol[i]);
        };
        bool last_dir, cur_dir;
        if (extr(0, last_dir)) return 0;
        int l = 0, r = n;
        while (l+1 < r) {
            int m = (l+r)/2;
            if (extr(m, cur_dir)) return m;
            bool rel_dir = cmp(pol[m], pol[l]);
            if ((!last_dir and cur_dir) or
                (last_dir == cur_dir and rel_dir == cur_dir)) {
                l = m;
                last_dir = cur_dir;
            } else r = m;
        }
        return l;
    }

    int max_dot(pt v) {
        return extreme([&](pt p, pt q) { return p*v > q*v; });
    }

    pair<int, int> tangents(pt p) {
        auto L = [&](pt q, pt r) { return ccw(p, r, q); };
        auto R = [&](pt q, pt r) { return ccw(p, q, r); };
        return {extreme(L), extreme(R)};
    }
}

```

```

}
};

bool operator <(const line& a, const line& b) { // comparador
    ↪ pra reta
    // assume que as retas tem p < q
    pt v1 = a.q - a.p, v2 = b.q - b.p;
    bool b1 = compare_angle(v1, v2), b2 = compare_angle(v2, v1);
    if (b1 or b2) return b1;
    return ccw(a.p, a.q, b.p); // mesmo angulo
}

bool operator ==(const line& a, const line& b) {
    return !(a < b) and !(b < a);
}

// comparador pro set pra fazer sweep line com segmentos
struct cmp_sweepline {
    bool operator () (const line& a, const line& b) const {
        // assume que os segmentos tem p < q
        if (a.p == b.p) return ccw(a.p, a.q, b.q);
        if (a.p.x != a.q.x and (b.p.x == b.q.x or a.p.x < b.p.x))
            return ccw(a.p, a.q, b.p);
        return ccw(a.p, b.q, b.p);
    }
};

// comparador pro set pra fazer sweep angle com segmentos
pt dir;
struct cmp_sweepangle {
    bool operator () (const line& a, const line& b) const {
        return get_t(dir, a) < get_t(dir, b);
    }
};

```

minkowski.h

Description: Soma de Minkowski. Computa $A+B = \{a+b : a \in A, b \in B\}$, onde A e B são polígonos convexos. $A+B$ tem no máximo $|A|+|B|$ pontos. $\text{dist}\{\}_{\text{convex}}(p, q)$ computa a distância mínima entre dois polígonos convexos.

Time: $O(|A| + |B|)$

```

vector<pt> minkowski(vector<pt> p, vector<pt> q) {
    auto fix = [](vector<pt>& P) {
        rotate(P.begin(), min_element(P.begin(), P.end()),
            ↪ P.end());
        P.push_back(P[0]), P.push_back(P[1]);
    };
    fix(p), fix(q);
    vector<pt> ret;
    int i = 0, j = 0;
    while (i < p.size()-2 or j < q.size()-2) {
        ret.push_back(p[i] + q[j]);
        auto c = ((p[i+1] - p[i]) ^ (q[j+1] - q[j]));
        if (c >= 0) i = min<int>(i+1, p.size()-2);
        if (c <= 0) j = min<int>(j+1, q.size()-2);
    }
    return ret;
}

ld dist_convex(vector<pt> p, vector<pt> q) {
    for (pt& i : p) i = i * -1;
    auto s = minkowski(p, q);
    if (inpol(s, pt(0, 0))) return 0;
    ld ans = DINF;
    for (int i = 0; i < s.size(); i++) ans = min(ans,
        disttoseg(pt(0, 0), line(s[(i+1)%s.size()], s[i])));
    return ans;
}

```

simplePoligon.h

Description: Verifica se um polígono com n pontos é simples (sem auto-interseções).

Time: $O(n \log n)$

```
bool operator < (const line& a, const line& b) { // comparador
    ↪ pro sweepline
    if (a.p == b.p) return ccw(a.p, a.q, b.q);
    if (!eq(a.p.x, a.q.x) and (eq(b.p.x, b.q.x) or a.p.x+eps <
    ↪ b.p.x))
        return ccw(a.p, a.q, b.p);
    return ccw(a.p, b.q, b.p);
}

bool simple(vector<pt> v) {
    auto intersects = [&](pair<line, int> a, pair<line, int> b)
    ↪ {
        if ((a.second+1)%v.size() == b.second or
            (b.second+1)%v.size() == a.second) return false;
        return interseg(a.first, b.first);
    };
    vector<line> seg;
    vector<pair<pt, pair<int, int>>> w;
    for (int i = 0; i < v.size(); i++) {
        pt at = v[i], nxt = v[(i+1)%v.size()];
        if (nxt < at) swap(at, nxt);
        seg.push_back(line(at, nxt));
        w.push_back({at, {0, i}});
        w.push_back({nxt, {1, i}});
        // casos degenerados estranhos
        if (isinseg(v[(i+2)%v.size()], line(at, nxt))) return 0;
        if (isinseg(v[(i+v.size()-1)%v.size()], line(at, nxt)))
            ↪ return 0;
    }
    sort(w.begin(), w.end());
    set<pair<line, int>> se;
    for (auto i : w) {
        line at = seg[i.second.second];
        if (i.second.first == 0) {
            auto nxt = se.lower_bound({at, i.second.second});
            if (nxt != se.end() and intersects(*nxt, {at,
            ↪ i.second.second})) return 0;
            if (nxt != se.begin() and intersects(*(--nxt), {at,
            ↪ i.second.second})) return 0;
            se.insert({at, i.second.second});
        } else {
            auto nxt = se.upper_bound({at, i.second.second}), cur =
            ↪ nxt, prev = --cur;
            if (nxt != se.end() and prev != se.begin()
                and intersects(*nxt, *(--prev))) return 0;
            se.erase(cur);
        }
    }
    return 1;
}
```

3 Strings

3.1 Number of Distinct Substrings ($O(n^2)$):

A classic solution is to iterate through all n suffixes $S[i..]$. For each suffix, we compute its Z-function in $O(n)$. The largest $Z[j]$ value found for this suffix represents the LCP (Longest Common Prefix) with another suffix $S[j..]$ that starts later. The number of *new* substrings introduced at i is the length of the suffix $(n - i)$ minus

the largest Z value found. The total sum gives us the number of distinct substrings.

3.2 Matching with $\leq k$ consecutive errors ($O(n)$):

To find a pattern P in a text T (length n) allowing a block of up to k consecutive errors, we use the Z-function twice.

1. Compute $Z(P + '$' + T)$ to find the LCP of P with each suffix $T[i..]$. This gives us $\text{lcp}[i]$, the length of the match *before* the first mismatch.
2. Compute $Z(P^R + '$' + T^R)$ (reversed strings). This allows us to find the LCS (Longest Common Suffix) of P ending at each position j in the text, let's call it $\text{lcs}[j]$.

A valid occurrence of P starts at $T[i]$ if the sum of the prefix match and the suffix match covers almost the entire pattern, i.e.: $\text{lcp}[i] + \text{lcs}[i + |P| - 1] \geq |P| - k$.

Hashing.h

Description: Hash polinomial de strings módulo MOD . Constrói em $O(n)$ e permite consultas em $O(1)$.

Usage: `getRange(a, b)` retorna o hash da substring $[a, b)$ (0-indexado).

Time: $O(n)$ build, $O(1)$ query

```
template<int MOD> struct Hashing{
    ll base, n;
    vector<ll> pow, ha;

    /*
    for random base:
    mt19937 rng((uint32_t)chrono::steady_clock::now().time_si_
    ↪ nce_epoch().count());
    const ll B = uniform_int_distribution<ll>(0, M - 1)(rng);
    */

    Hashing(string & s, int a) : n(s.size()), base(a)
    ↪ , pow(n+1), ha(n+1){

        pow[0] = 1;
        for(int i = 0; i < n; i++){
            ha[i+1] = (ha[i] * base + s[i])%MOD;
            pow[i+1] = (pow[i] * base)%MOD;
        }

        int getRange(int a, int b){
            assert(a <= b);
            ll hash = (ha[b] - (ha[a] * pow[b-a])%MOD)%MOD;
            return hash < 0 ? hash + MOD : hash;
        }
    };
};
```

kmp.h

Description: KMP (Knuth-Morris-Pratt). Encontra padrões em strings.

Usage: `find_pi(s)` retorna a função prefixo. `kmp(s, t)` retorna as posições de ocorrência de s em t . `autKMP` constrói o autômato do KMP.

Time: $O(n)$ `find_pi`, $O(n + m)$ `kmp`

```
vector<int> find_pi(string s){
    vector<int> pi(s.size());
    for(int i = 1, j = 0; i < s.size(); i++){
        while(j > 0 && s[j] != s[i]) j = pi[j-1];
        if(s[j] == s[i]) j++;
        pi[i] = j;
    }
    return pi;
};

vector<int> kmp(string s, string t){
    vector<int> pi= find_pi(s), match;
    for(int i = 0, j = 0; i < t.size(); i++){
        while(j > 0 && t[i] != s[j]) j = pi[j-1];
        if(t[i] == s[j]) j++;
        if(j == s.size()) {match.push_back(i-j+1); j =
        ↪ pi[j-1];}
    }
    return match;
};

struct autKMP {
    vector<vector<int>> nxt;

    autKMP(string& s) : nxt(26, vector<int>(s.size()+1)) {
        vector<int> p = pi(s);
        nxt[s[0]-'a'][0] = 1;
        for (char c = 0; c < 26; c++)
            for (int i = 1; i <= s.size(); i++)
                nxt[c][i] = c == s[i]-'a' ? i+1 :
                ↪ nxt[c][p[i-1]];
    }
};
```

manacher.h

Description: Algoritmo de Manacher. Encontra o tamanho dos palíndromos em uma string.

Usage: `manacher_odd(s)` para palíndromos ímpares. `manacher(s)` retorna $\{d_{\text{odd}}, d_{\text{even}}\}$ para todos os centros.

Time: $O(n)$, onde n é o tamanho da string

```
vector<int> manacher_odd(string s) {
    int n = s.size();
    s = "$" + s + "~";
    vector<int> p(n + 2);
    int l = 0, r = 1;
    for(int i = 1; i <= n; i++) {
        p[i] = min(r - i, p[l + (r - i)]);
        while(s[i - p[i]] == s[i + p[i]]) {
            p[i]++;
        }
        if(i + p[i] > r) {
            l = i - p[i], r = i + p[i];
        }
    }
    return vector<int>(begin(p) + 1, end(p) - 1);
}

pair<vector<int>, vector<int>> manacher(string s) {
    string t;
    for(auto c: s) {
        t += string("#") + c;
    }
    vector<int> res = manacher_odd(t + "#");
    vector<int> dodd(s.size()), deven(s.size());
    for(int i = 0; i < s.size(); i++){
```

```

    dodd[i] = res[2*i + 1]/2;
    deven[i] = (res[2*i]-1)/2;
}

return {dodd, deven};
}

```

SuffixArray.h

Description: Suffix Array $O(n \log n)$. suffix_array(s) retorna o SA. kasai(s, sa) calcula lcp[i] = LCP entre $s[sa[i]..]$ e $s[sa[i+1]..]$. LCP entre sufixos i e j: min do lcp no intervalo apropriado.

Usage: suffix_array(s) constrói o SA. kasai(s, sa) constrói o array LCP.

Time: $O(n \log n)$ suffix_array, $O(n)$ kasai

```

vector<int> suffix_array(string s) {
    s += "$";
    int n = s.size(), N = max(n, 260);
    vector<int> sa(n), ra(n);
    for(int i = 0; i < n; i++) sa[i] = i, ra[i] = s[i];

    for(int k = 0; k < n; k ? k *= 2 : k++) {
        vector<int> nsa(sa), nra(n), cnt(N);

        for(int i = 0; i < n; i++) nsa[i] = (nsa[i]-k+n)%n,
            cnt[ra[i]]++;
        for(int i = 1; i < N; i++) cnt[i] += cnt[i-1];
        for(int i = n-1; i+1; i--) sa[--cnt[ra[nsa[i]]]] = nsa[i];

        for(int i = 1, r = 0; i < n; i++) nra[sa[i]] = r +=
            ra[sa[i]] !=
            ra[sa[i-1]] or ra[(sa[i]+k)%n] != ra[(sa[i-1]+k)%n];
        ra = nra;
        if (ra[sa[n-1]] == n-1) break;
    }
    return vector<int>(sa.begin()+1, sa.end());
}

vector<int> kasai(string s, vector<int> sa) {
    int n = s.size(), k = 0;
    vector<int> ra(n), lcp(n);
    for (int i = 0; i < n; i++) ra[sa[i]] = i;

    for (int i = 0; i < n; i++, k -= !!k) {
        if (ra[i] == n-1) { k = 0; continue; }
        int j = sa[ra[i]+1];
        while (i+k < n and j+k < n and s[i+k] == s[j+k]) k++;
        lcp[ra[i]] = k;
    }
    return lcp;
}

```

SuffixArray2.h

Description: Suffix Array $O(n)$ (DC3 / SA-IS). Computa sa, rnk, lcp.

Usage: query(i, j) retorna o LCP entre $s[i..n-1]$ e $s[j..n-1]$ em $O(1)$. count_substr(t) retorna quantas vezes t ocorre em s. sos() computa $\sum_i f(\text{ocorrências de } t)$ para toda substring distinta t.

Time: $O(n)$ build, $O(1)$ query

```

template<typename T> struct rmq {
    vector<T> v;
    int n; static const int b = 30;
    vector<int> mask, t;

    int op(int x, int y) { return v[x] <= v[y] ? x : y; }
    int msb(int x) { return __builtin_clz(1)-__builtin_clz(x); }
}

```

```

int small(int r, int sz = b) { return
    r-msb(mask[r]&((1<<sz)-1)); }
rmq() {}
rmq(const vector<T>& v_) : v(v_), n(v.size()), mask(n),
    t(n) {
    for (int i = 0, at = 0; i < n; mask[i++] = at |= 1) {
        at = (at<<1)&((1<<b)-1);
        while (at and op(i-msb(at&-at), i) == i) at ^= at&-at;
    }
    for (int i = 0; i < n/b; i++) t[i] = small(b*i+b-1);
    for (int j = 1; (1<<j) <= n/b; j++) for (int i = 0;
        i+(1<<j) <= n/b; i++)
        t[n/b*j+i] = op(t[n/b*(j-1)+i],
            t[n/b*(j-1)+i+(1<<(j-1))]);
}

int index_query(int l, int r) {
    if (r-l+1 <= b) return small(r, r-l+1);
    int x = l/b+1, y = r/b-1;
    if (x > y) return op(small(l+b-1), small(r));
    int j = msb(y-x+1);
    int ans = op(small(l+b-1), op(t[n/b*j+x],
        t[n/b*j+y-(1<<j)+1]));
    return op(ans, small(r));
}

T query(int l, int r) { return v[index_query(l, r)]; }
};

struct suffix_array {
    string s;
    int n;
    vector<int> sa, cnt, rnk, lcp;
    rmq<int> RMQ;

    bool cmp(int a1, int b1, int a2, int b2, int a3=0, int
        b3=0) {
        return a1 != b1 ? a1 < b1 : (a2 != b2 ? a2 < b2 : a3 <
            b3);
    }

    template<typename T> void radix(int* fr, int* to, T* r, int
        N, int k) {
        cnt = vector<int>(k+1, 0);
        for (int i = 0; i < N; i++) cnt[r[fr[i]]]++;
        for (int i = 1; i <= k; i++) cnt[i] += cnt[i-1];
        for (int i = N-1; i+1; i--) to[--cnt[r[fr[i]]]] = fr[i];
    }

    void rec(vector<int>& v, int k) {
        auto &tmp = rnk, &m0 = lcp;
        int N = v.size()-3, sz = (N+2)/3, sz2 = sz+N/3;
        vector<int> R(sz2+3);
        for (int i = 1, j = 0; j < sz2; i += i%3) R[j++] = i;

        radix(&R[0], &tmp[0], &v[0]+2, sz2, k);
        radix(&tmp[0], &R[0], &v[0]+1, sz2, k);
        radix(&R[0], &tmp[0], &v[0]+0, sz2, k);

        int dif = 0;
        int l0 = -1, l1 = -1, l2 = -1;
        for (int i = 0; i < sz2; i++) {
            if (v[tmp[i]] != l0 or v[tmp[i]+1] != l1 or v[tmp[i]+2]
                != l2)
                l0 = v[tmp[i]], l1 = v[tmp[i]+1], l2 = v[tmp[i]+2],
                dif++;
            if (tmp[i]%3 == 1) R[tmp[i]/3] = dif;
            else R[tmp[i]/3+sz] = dif;
        }

        if (dif < sz2) {
            rec(R, dif);
            for (int i = 0; i < sz2; i++) R[sa[i]] = i+1;
        }
}

```

```

} else for (int i = 0; i < sz2; i++) sa[R[i]-1] = i;

for (int i = 0, j = 0; j < sz2; i++) if (sa[i] < sz)
    tmp[j++] = 3*sa[i];
radix(&tmp[0], &m0[0], &v[0], sz, k);
for (int i = 0; i < sz2; i++)
    sa[i] = sa[i] < sz ? 3*sa[i]+1 : 3*(sa[i]-sz)+2;

int at = sz2+sz-1, p = sz-1, p2 = sz2-1;
while (p >= 0 and p2 >= 0) {
    if ((sa[p2]%3==1 and cmp(v[m0[p]], v[sa[p2]]),
        R[m0[p]/3],
        R[sa[p2]/3+sz])) or (sa[p2]%3==2 and cmp(v[m0[p]],
            v[sa[p2]],
            v[m0[p]+1], v[sa[p2]+1], R[m0[p]/3+sz],
            R[sa[p2]/3+1]))
        sa[at--] = sa[p2--];
    else sa[at--] = m0[p--];
}
while (p >= 0) sa[at--] = m0[p--];
if (N%3==1) for (int i = 0; i < N; i++) sa[i] = sa[i+1];
}

suffix_array(const string& s_) : s(s_), n(s.size()),
    sa(n+3),
    cnt(n+1), rnk(n), lcp(n-1) {
    vector<int> v(n+3);
    for (int i = 0; i < n; i++) v[i] = i;
    radix(&v[0], &rnk[0], &s[0], n, 256);
    int dif = 1;
    for (int i = 0; i < n; i++)
        v[rnk[i]] = dif += (i and s[rnk[i]] != s[rnk[i-1]]);
    if (n >= 2) rec(v, dif);
    sa.resize(n);

    for (int i = 0; i < n; i++) rnk[sa[i]] = i;
    for (int i = 0, k = 0; i < n; i++, k -= !!k) {
        if (rnk[i] == n-1) {
            k = 0;
            continue;
        }
        int j = sa[rnk[i]+1];
        while (i+k < n and j+k < n and s[i+k] == s[j+k]) k++;
        lcp[rnk[i]] = k;
    }
    RMQ = rmq<int>(lcp);
}

int query(int i, int j) {
    if (i == j) return n-i;
    i = rnk[i], j = rnk[j];
    return RMQ.query(min(i, j), max(i, j)-1);
}

pair<int, int> next(int L, int R, int i, char c) {
    int l = L, r = R+1;
    while (l < r) {
        int m = (l+r)/2;
        if (i+sa[m] >= n or s[i+sa[m]] < c) l = m+1;
        else r = m;
    }
    if (l == R+1 or s[i+sa[l]] > c) return {-1, -1};
    L = l;

    l = L, r = R+1;
    while (l < r) {
        int m = (l+r)/2;
        if (i+sa[m] >= n or s[i+sa[m]] <= c) l = m+1;
        else r = m;
    }
    R = l-1;
}

```

```

    return {L, R};
}
// quantas vezes 't' ocorre em 's' - 0(|t| log n)
int count_substr(string& t) {
    int L = 0, R = n-1;
    for (int i = 0; i < t.size(); i++) {
        tie(L, R) = next(L, R, i, t[i]);
        if (L == -1) return 0;
    }
    return R-L+1;
}

// exemplo de f que resolve o problema
// https://codeforces.com/edu/course/2/5/practice/
↪ contest/269656/problem/D
ll f(ll k) { return k*(k+1)/2; }

ll dfs(int L, int R, int p) { // dfs na suffix tree chamado
    ↪ em pre ordem
    int ext = L != R ? RMQ.query(L, R-1) : n - sa[L];

    // Tem 'ext - p' substrings diferentes que ocorrem 'R-L+1'
    ↪ vezes
    // 0 LCP de todas elas eh 'ext'
    ll ans = (ext-p)*f(R-L+1);

    // L eh terminal, e folha sse L == R
    if (sa[L]+ext == n) L++;

    // se for um SA de varias strings separadas como s#t$u&,
    ↪ usar no lugar do if de cima
    // (separadores < 'a', diferentes e inclusive no final)
    // while (L <= R && (sa[L]+ext == n || s[sa[L]+ext] <
    ↪ 'a')) {
    //     L++;
    // }

    while (L <= R) {
        int idx = L != R ? RMQ.index_query(L, R-1) : -1;
        if (idx == -1 || lcp[idx] != ext) idx = R;

        ans += dfs(L, idx, ext);
        L = idx+1;
    }
    return ans;
}

// sum over substrings: computa, para toda substring t
↪ distinta de s,
// \sum f(# ocorrencias de t em s) - 0 (n)
ll sos() { return dfs(0, n-1, 0); }
};

```

trie.h

Description: Trie para strings. Suporta alfabeto customizável via parâmetros sigma e norm.

Usage: insert(s) adiciona string, erase(s) remove, find(s) retorna posição (-1 se não achar), count_pref(s) conta quantas strings têm s como prefixo.

Time: $O(|s| \cdot \sigma)$ por operação

```

struct trie {
    vector<vector<int>>> to;
    vector<int> end, pref;
    int sigma; char norm;
    trie(int sigma_=26, char norm_='a') : sigma(sigma_),
    ↪ norm(norm_) {
        to = {vector<int>(sigma)};
        end = {0}, pref = {0};
    }
};

```

```

}
void insert(string s) {
    int x = 0;
    for (auto c : s) {
        int &nxt = to[x][c-norm];
        if (!nxt) {
            nxt = to.size();
            to.push_back(vector<int>(sigma));
            end.push_back(0), pref.push_back(0);
        }
        x = nxt, pref[x]++;
    }
    end[x]++, pref[0]++;
}
void erase(string s) {
    int x = 0;
    for (char c : s) {
        int &nxt = to[x][c-norm];
        x = nxt, pref[x]--;
        if (!pref[x]) nxt = 0;
    }
    end[x]--, pref[0]--;
}
int find(string s) {
    int x = 0;
    for (auto c : s) {
        x = to[x][c-norm];
        if (!x) return -1;
    }
    return x;
}
int count_pref(string s) {
    int id = find(s);
    return id >= 0 ? pref[id] : 0;
}
};

```

z.h

Description: Z-function. $Z[i]$ é o tamanho do maior prefixo da string que começa na posição i e é igual a um prefixo da string original.

Time: $O(n)$

```

vector<int> zfunc(string s){
    int n = s.size();
    vector<int> z(n);
    for(int i = 1, l = 0, r = 0; i < n; i++){
        if(i <= r) z[i] = min(z[i-l], r-i+1);
        while(i + z[i] < n && s[i + z[i]] == s[z[i]]){
            z[i]++;
        }
        if(i+z[i]-1 > r){
            r = i+z[i]-1;
            l = i;
        }
    }
    return z;
}

```

4 Grafos

4.1 Min Cut Max Flow Duality

We seek to construct a binary string S of length n that minimizes a total cost. The cost is defined as follows:

- If $S_i = 0$, a cost of A_i is incurred.

- If $S_i = 1$, a cost of B_i is incurred.

- If $S_i = 1$ and $S_j = 0$, a penalty of $C_{i,j}$ is incurred.

The total cost is the sum of all such costs and penalties for the chosen string S .

1. For each node i , add an edge $s \rightarrow i$ with capacity B_i . This represents the cost of setting $S_i = 1$.
2. For each node i , add an edge $i \rightarrow t$ with capacity A_i . This represents the cost of setting $S_i = 0$.
3. For each pair (i, j) with a penalty, add an edge $i \rightarrow j$ with capacity $C_{i,j}$. This represents the penalty for setting $S_i = 1$ and $S_j = 0$.

The capacity of a cut in this graph corresponds to the total cost of the binary string defined by the partition. For example, if node i is in the T -partition ($S_i = 1$) and node j is in the S -partition ($S_j = 0$), the edge $i \rightarrow j$ must be cut, adding the penalty $C_{i,j}$ to the total cost. By the max-flow min-cut theorem, the minimum cost is equal to the maximum flow from s to t .

4.2 Notable Applications and Equivalences on Flow

• Bipartite Matching:

The size of the maximum matching in a bipartite graph is equal to the maximum flow in a network constructed from the graph.

• König's Theorem:

In any bipartite graph, the number of edges in the maximum matching is equal to the number of vertices in the minimum vertex cover.

$$\text{Maximum Matching} = \text{Minimum Vertex Cover}$$

$$\text{Maximum Independent Set} = |V| - \text{Maximum Matching}$$

• Menger's Theorem:

The maximum number of vertex-disjoint paths between two vertices u, v is equal to the minimum number of vertices to be removed to disconnect u and v .

• Project Selection Problem (Min-Cut):

Binary decision problems with interdependent costs and profits can be modeled as a minimum cut problem, where the cut separates the "chosen" decisions from the "not chosen" ones.

4.3 Game Theory

Impartial Games (Normal Play): Último a jogar vence.

- **Nim-Sum:** $G = p_1 \oplus p_2 \oplus \dots \oplus p_k$.
- **P-position (Losing):** $G = 0$. **N-position (Winning):** $G \neq 0$.
- **Sprague-Grundy:** Qualquer estado S equivale a uma pilha de Nim de tamanho $g(S)$.
- **Cálculo de $g(S)$:** $g(S) = \text{mex}(\{g(S') \mid S \rightarrow S'\})$.
- **Soma de Jogos:** $g(G_1 + \dots + G_k) = g(G_1) \oplus \dots \oplus g(G_k)$.
- **Movimento de Divisão:** Se uma jogada divide o jogo em subjogos A e B , o valor de Grundy dessa jogada é $g(A) \oplus g(B)$. O $g(S)$ final será o MEX de todos os valores de jogadas possíveis.
- **Subtraction Games:** $g(n) = \text{mex}(\{g(n - s_i) \mid s_i \in S\})$. Costuma ser periódico; procure o ciclo para n grandes.
- **Misere Play:** Último a jogar PERDE. No Nim, jogue normalmente até que reste apenas uma pilha de tamanho > 1 . Nesse momento, mova para deixar um número ÍMPAR de pilhas de tamanho 1.

Green Hackenbush:

- **Em Árvore:** $g(\text{node}) = \bigoplus (g(\text{filho}) + 1)$.
- **Colon Principle:** Quando arestas se encontram em um vértice, substitua por uma única aresta com valor $g = \text{XOR}$ dos Grundy das arestas originais.

2Sat.h

Description: 2-SAT solver. Resolve equações do tipo $(a \vee b) \wedge (c \vee d) \dots$

Cláusula $(+a \vee -b)$ vira `\{\}texttt{add\{\}_edge(1, a, 0, b)}`.

Usage: `\{\}texttt{add\{\}_edge(1, a, 0, b)}` para $(+a \vee -b)$.

Time: $O(n + m)$, onde n = variáveis, m = implicações

```
struct SAT2{
    int n, cont;
    vector<char> resp;
    vector<int> marc, ord, comp;
    vector<vector<int>> grafo, rgrafo, scc;

    SAT2(int n) : n(n), marc(2*n+2), grafo(2*n+2),
        ↪ rgrafo(2*n+2), comp(2*n+2), resp(2*n + 2){

    void add_edge(int sx, int x, int sy, int y){ // '+' = 1,
        ↪ '-' = 0

        grafo[y+n*(!sy)].push_back(x+n*sx); //~y -> x
        grafo[x+n*(!sx)].push_back(y+n*sy); //~x -> y

        rgrafo[x+n*sx].push_back(y+n*(!sy));
        rgrafo[y+n*sy].push_back(x+n*(!sx));

    }

    void dfs1(int v){
        marc[v] = 1;
        for(auto viz : grafo[v]){
            if(!marc[viz]) dfs1(viz);
        }
        ord.push_back(v);
    }
};
```

```
void dfs2(int v, int c){
    comp[v] = c;
    for(auto viz : rgrafo[v]){
        if(!comp[viz]) dfs2(viz, c);
    }
}

void build(){
    cont = 0;

    for(int i = 1; i <= 2*n; i++){
        if(!marc[i]) dfs1(i);
    }

    reverse(ord.begin(), ord.end());

    for(int v : ord){
        if(!comp[v]){
            dfs2(v, ++cont);
        }
    }

    bool can = true;
    for(int i = 1; i <= n; i++){
        if(comp[i] == comp[i+n]) can = false;
        resp[i] = comp[i] < comp[i+n] ? '+' : '-';
        //positiva é comp[i+n], está escolhendo a variavel
        ↪ que não
        //tem um caminho de implicação que resulta em
        ↪ impossível
    }

    if(can){
        for(int i = 1; i <= n; i++){
            cout << resp[i] << " ";
        }
    }
    else{
        cout << "IMPOSSIBLE" << endl;
    }
}

};
```

ArtPontes.h

Description: Pontes e Articulação. Encontra pontos de articulação e pontes em um grafo não direcionado.

Time: $O(V + E)$

```
struct ArticPont{
    int n;
    int count = 0;
    vector<int> marc, tin, low, artic;
    vector<vector<int>> grafo;
    vector<pair<int, int>> bridges;

    ArticPont(int n) : n(n), grafo(n+1), marc(n+1), tin(n+1),
        ↪ low(n+1), artic(n+1) {}

    void add_edge(int a, int b){
        grafo[a].push_back(b);
        grafo[b].push_back(a);
    }

    void dfs(11 x, 11 p){
        marc[x] = 1;
        tin[x] = low[x] = ++count;
        11 children = 0;
```

```
for(11 viz : grafo[x]){
    if(viz == p) continue;
    if(marc[viz]){
        low[x] = min(low[x], tin[viz]);
    }else{
        dfs(viz, x);
        low[x] = min(low[x], low[viz]);
        if(low[viz] > tin[x]){
            bridges.push_back({min(viz, x), max(viz,
                ↪ x)});
        }
        if(low[viz] >= tin[x] && p) artic[x] = 1;
        children++;
    }
}
if(!p && children > 1) artic[x] = 1;
}

void find_brig_and_artc(){
    for(11 i=1; i<=n; i++){
        if(!marc[i]) dfs(i, 0);
    }
}

};
```

BellmanFord.h

Description: Bellman-Ford para caminhos mínimos. Para encontrar os ciclos negativos basta guardar os pais de cada vértice.

Time: $O(nm)$, onde n = vértices, m = arestas

```
struct Edge{
    int v, u, cost;
    Edge(int v, int u, int cost): v(v), u(u), cost(cost) {}
};

struct Ford
{
    const 11 INFL = 1e18;
    int n, m;
    vector<Edge> edges;
    vector<11> dist;

    Ford(int n, int m) : n(n), m(m), dist(n+1, INFL) {}

    void add_edge(int v, int u, int cost){
        edges.emplace_back(v, u, cost);
    }

    11 bellman(int s, int t){
        dist[s] = 0;

        //Encontrar distancias
        for(int k=1; k < n; k++){
            for(Edge e : edges){
                int a = e.v, b = e.u, c = e.cost;
                if(dist[a] != MINFL && dist[b] > dist[a] + c){
                    dist[b] = dist[a] + c;
                }
            }
        }

        //Se conseguirmos melhorar após n-1, significa que
        ↪ existe ciclo negativo
        for(Edge e : edges){
            int a = e.v, b = e.u, c = e.cost;
            if(dist[a] != MINFL && dist[b] > dist[a] + c){
                return -1;
            }
        }
    }
};
```

```

    }
    return dist[t];
}

};

```

Bfs.h

Description: BFS porque o caio não sabe codar
Time: $O(N)$

```

queue<int> q;
vector<bool> used(n);

q.push(s);
used[s] = true;
while (!q.empty()) {
    int v = q.front();
    q.pop();
    for (int u : adj[v]) {
        if (!used[u]) {
            used[u] = true;
            q.push(u);
        }
    }
}

```

BridgeTree.h

Description: Bridge Tree. Encontra a árvore de pontes de um grafo, onde cada aresta da árvore representa uma ponte do grafo original.
Time: $O(V + E)$

```

struct BridgeTree{
    int n;
    int count = 0;
    vector<int> marc, tin, low, is_bridge;
    vector<vector<pair<int,int>>> grafo;
    vector<vector<int>> BT;
    vector<pair<int,int>> edge;

    vector<int> BTcomponent;

    BridgeTree(int n) : n(n), grafo(n+1), marc(n+1),
        tin(n+1), low(n+1), BTcomponent(n+1){}

    void add_edge(int a, int b){
        grafo[a].push_back({b, edge.size()});
        grafo[b].push_back({a, edge.size()});
        edge.push_back({a,b});
        is_bridge.push_back(0);
    }

    void dfs(int x, int p){
        marc[x] = 1;
        tin[x] = low[x] = ++count;
        int children = 0;
        for(auto [viz, e] : grafo[x]){
            if(viz == p) continue;
            if(marc[viz]){
                low[x] = min(low[x], tin[viz]);
            }else{
                dfs(viz,x);
                low[x] = min(low[x], low[viz]);
                if(low[viz] > tin[x]){

```

```

                    is_bridge[e] = 1;
                }
                children++;
            }
        }

        void find_bridges(){
            for(ll i=1; i<=n; i++){
                if(!marc[i]) dfs(i,0);
            }
        }

        void BTdfs(int v, int comp){
            BTcomponent[v] = comp;
            for(auto [viz, e] : grafo[v]){
                if(BTcomponent[viz] || is_bridge[e]) continue;
                BTdfs(viz, comp);
            }
        }

        void BrigeTree(){
            int comp = 0;
            for(int i = 1; i <= n; i++){
                if(!BTcomponent[i]) BTdfs(i, ++comp);
            }

            BT.resize(comp+1);

            for(int i = 1; i <= n; i++){
                for(auto [j,e] : grafo[i]){
                    if(is_bridge[e]){
                        BT[BTcomponent[i]].push_back(BTcomponent[j]
                            ← j]);
                        BT[BTcomponent[j]].push_back(BTcomponent[i]
                            ← i]);
                    }
                }
            }
        }
};

```

Dijkstra.h

Description: Algoritmo de caminho mínimo para grafos com pesos não negativos. De uma fonte para todos.
Time: $O(M + N \log N)$

```

struct Dykstra
{
    ll INF = 1e18;

    int n;
    vector<ll> dist;
    vector<vector<pair<int,int>>> g;

    Dykstra(int n) : n(n), dist(n+1,INF), g(n+1) {}

    void addEdge(ll v, ll u, ll p){
        g[v].push_back({u,p});
        g[u].push_back({v,p});
    }

    void run(ll v){
        priority_queue<pair<ll,ll>, vector<pair<ll,ll>>,
            greater<pair<ll,ll>>> fila;

        fila.push({0,v});

```

```

        while (!fila.empty())
        {
            ll vert = fila.top().second;
            ll price = fila.top().first;
            fila.pop();

            if(dist[vert] != INF) continue;

            dist[vert] = price;

            for(auto viz : g[vert]){
                ll nxt = viz.first;
                ll cost = viz.second;
                fila.push({price + cost, nxt});
            }
        }
    }
};

```

Dijkstra2.h

Description: Dijkstra $O(n^2 + m)$ para grafos sem pesos negativos. De uma fonte para todos. Implementação sem priority queue (densa).
Time: $O(n^2 + m)$

```

struct Graph{
    int n;
    const int binf = 1e9;
    vector <vector<pair<int,int>>> adj;
    Graph(int n){
        this->n = n;
        adj.resize(n);
    }
    void add_edge(int u, int v, int w){
        adj[u].emplace_back(v, w);
        adj[v].emplace_back(u, w);
    }
    int min_dist(int s, int e){//disjktras da silva
        vector <int> d(n, binf);
        vector <bool> u(n, false);
        d[s] = 0;
        for(int i = 0; i < n; i++){
            int v = -1;
            for(int j = 0; j < n; j++){
                if(!u[j] && (v==-1 || d[j] < d[v])){
                    v = j;
                }
            }
            if(d[v] == binf){
                break;
            }
            u[v] = true;
            for(auto edge : adj[v]){
                int to = edge.first;
                int len = edge.second;
                if(d[v]+len < d[to]){
                    d[to] = d[v]+len;
                }
            }
        }
        return d[e];
    }
};

```

Dinic.h

Description: Dinic para fluxo máximo. Grafo com capacidades 1: $O(\min(M\sqrt{M}, MN^{2/3}))$. Todo vértice tem grau de entrada ou saída 1 e a maior capacidade é 1: $O(\sqrt{N} \cdot M)$ \{\}texttt{recap()} retorna as arestas do min-cut.

Time: $O(\min(V \cdot \max_flow, V^2 E))$

```
template<typename T>
struct Dinic{
    struct Edge {int v, u; T cap, flow;};
    int m=0;
    vector<Edge> edges;
    vector<vector<int>> > vec;
    vector<int> lv, pos;
    queue<int> fila;

    Dinic() {}

    Dinic(int n) : vec(n), lv(n), pos(n) {}

    void add_edge(int v, int u, T cap) {
        edges.push_back({v, u, cap, 0});
        edges.push_back({u, v, 0, 0});
        vec[v].push_back(m);
        vec[u].push_back(m+1);
        m+=2;
    }

    int bfs(int t){
        while(!fila.empty()){
            int v=fila.front();
            fila.pop();
            for(int i:vec[v]){
                if(edges[i].cap-edges[i].flow<1) continue;
                if(lv[edges[i].u]!=-1) continue;

                lv[edges[i].u]=lv[v]+1;
                fila.push(edges[i].u);
            }
        }
        return lv[t]!=-1;
    }

    T dfs(int v, int t, T menor) {
        if(!menor) return 0;
        if(v==t) return menor;

        for(int& j=pos[v]; j<(int)vec[v].size(); j++){
            int i=vec[v][j];
            int u=edges[i].u;

            if(lv[v]+1!=lv[u] ||
                ↪ edges[i].cap-edges[i].flow<1) continue;

            T agr=dfs(u, t, min(menor,
                ↪ edges[i].cap-edges[i].flow));
            if(!agr) continue;

            edges[i].flow+=agr;
            edges[i~1].flow-=agr;

            return agr;
        }
        return 0;
    }

    T max_flow(int s, int t){
        T flow=0;
        while(1){
            fill(lv.begin(), lv.end(), -1);
```

```
            lv[s]=0;
            fila.push(s);

            if(!bfs(t)) break;

            fill(pos.begin(), pos.end(), 0);

            while(T atual=dfs(s, t, INF)) flow+=atual;
            ↪ //remember to change INF
        }
        return flow;
    }

    auto recap(){
        vector<pair<int, int> > resp;
        for(int i=0; i<(int)edges.size(); i+=2){
            if(lv[edges[i].v]>=0 && lv[edges[i].u]==-1)
                ↪ resp.push_back({edges[i].v, edges[i].u});
        }
        return resp;
    }
};

DirectedMST.h

Description: Directed MST (Minimum Spanning Arborescence). Retorna o
pai de cada vértice na arborescência mínima enraizada em \{\}texttt{root}.
Assume que existe solução (todos vértices alcançáveis de \{\}texttt{root}).
Cada aresta deve ser distinta. 0-indexed. ~300ms para  $n = 2 \cdot 10^5$ .
Time:  $O(E \log V)$ 

const int N = 3e5 + 9;
const long long inf = 1e18;

template<typename T> struct PQ {
    long long sum = 0;
    priority_queue<T, vector<T>, greater<T>> Q;
    void push(T x) { x.w -= sum; Q.push(x); }
    void pop() { auto ans = Q.top(); Q.pop(); ans.w += sum; return
        ↪ ans; }
    int size() { return Q.size(); }
    void add(long long x) { sum += x; }
    void merge(PQ &&x) {
        if (size() < x.size()) swap(sum, x.sum), swap(Q, x.Q);
        while (x.size()) {
            auto tmp = x.pop();
            tmp.w -= sum;
            Q.push(tmp);
        }
    }
};

struct edge {
    int u, v; long long w;
    bool operator > (const edge &rhs) const { return w > rhs.w;
        ↪ }
};

struct DSU {
    vector<int> par;
    DSU(int n) : par(n, -1) {}
    int root(int i) { return par[i] < 0 ? i : par[i] =
        ↪ root(par[i]); }
    void set_par(int c, int p) { par[c] = p; }
};

// returns parents of each vertex
// each edge should be distinct
// it assumes that a solution exists (all vertices are
    ↪ reachable from root)
// 0 indexed
```

```
// Takes ~300ms for n = 2e5
vector<int> DMST(int n, int root, const vector<edge> &edges) {
    vector<int> u(2 * n - 1, -1), par(2 * n - 1, -1);
    edge par_edge[2 * n - 1];
    vector<int> child[2 * n - 1];

    PQ<edge> Q[2 * n - 1];
    for (auto e : edges) Q[e.v].push(e);
    for (int i = 0; i < n; i++) Q[(i + 1) % n].push({i, (i + 1)
        ↪ % n, inf});

    int super = n;
    DSU dsu(2 * n - 1);
    int head = 0;
    while (Q[head].size()) {
        auto x = Q[head].pop();
        int nxt_root = dsu.root(x.u);
        if (nxt_root == head) continue;
        u[head] = nxt_root;
        par_edge[head] = x;
        if (u[nxt_root] == -1) head = nxt_root;
        else {
            int j = nxt_root;
            do {
                Q[j].add(-par_edge[j].w);
                Q[super].merge(move(Q[j]));
                assert(u[j] != -1);
                child[super].push_back(j);
                j = dsu.root(u[j]);
            } while (j != nxt_root);
            for (auto u : child[super]) par[u] = super,
                ↪ dsu.set_par(u, super);
            head = super++;
        }
    }
    vector<int> res(2 * n - 1, -1);
    queue<int> q; q.push(root);
    while (q.size()) {
        int u = q.front();
        q.pop();
        while (par[u] != -1) {
            for (auto v : child[par[u]]) {
                if (v != u) {
                    res[par_edge[v].v] = par_edge[v].u;
                    q.push(par_edge[v].v);
                    par[v] = -1;
                }
            }
            u = par[u];
        }
    }
    res[root] = root; res.resize(n);
    return res;
}

int32_t main() {
    ios_base::sync_with_stdio(0);
    cin.tie(0);
    int n, m, root; cin >> n >> m >> root;
    vector<edge> edges(m);
    for (auto &e : edges) cin >> e.u >> e.v >> e.w;
    auto res = DMST(n, root, edges);

    unordered_map<int, int> W[n];
    for (auto u : edges) W[u.v][u.u] = u.w;

    long long ans = 0;
    for (int i = 0; i < n; i++) if (i != root) ans +=
        ↪ W[i][res[i]];
    cout << ans << '\n';
}
```

```
for (auto x : res) cout << x << ' '; cout << '\n';
return 0;
}
```

EulerPath.h

Description: Eulerian undirected/directed path/cycle algorithm. Input should be a vector of (dest, global edge index), where for undirected graphs, forward/backward edges have the same index. Returns a list of nodes in the Eulerian path/cycle with src at both start and end, or empty list if no cycle/path exists. To get edge indices back, add .second to s and ret.

Time: $O(V + E)$

```
vector<int> eulerWalk(vector<vector<pii>>& gr, int nedges,
↳ int src=0) {
    int n = gr.size();
    vector<int> D(n), its(n), eu(nedges), ret, s = {src};
    D[src]++; // to allow Euler paths, not just cycles
    while (!s.empty()) { //start-hash
        int x = s.back(), y, e, &it = its[x], end =
        ↳ int(gr[x].size());
        if (it == end){ ret.push_back(x); s.pop_back(); continue;
        ↳ }
        tie(y, e) = gr[x][it++];
        if (!eu[e])
            D[x]--, D[y]++, eu[e] = 1, s.push_back(y);
    } //end-hash
    for(auto &x : D) if (x < 0 || int(ret.size()) != nedges+1)
        ↳ return {};
    return {ret.rbegin(), ret.rend()};
}
```

FloydWarshall.h

Description: Floyd-Warshall para caminhos mínimos entre todos os pares. Se houver ciclos negativos, para algum vértice $a \rightarrow dist[a][a] < 0$.

Time: $O(n^3)$

```
struct FloydWarshall
{
    const int MAXN = 500;
    const ll INF = 1e18;
    vector<dist>(maxn, vector<ll>(maxn, INF));

    void floydWarshall() {
        for(int i = 0; i < MAXN; i++) dist[i][i] = 0;

        for(int k = 1; k < MAXN; k++)
            for(int i = 1; i < MAXN; i++)
                for(int j = 1; j < MAXN; j++){
                    if(dist[i][k] < INF && dist[k][j] < INF)
                        dist[i][j] = min(dist[i][j],
                            ↳ dist[i][k] + dist[k][j]);
                }
    }
};
```

Games.h

Description: Jogos em grafos arbitrários. Determina estados winning/losing em jogos em grafos arbitrários usando backward induction.

Time: $O(V + E)$

```
vector<vector<int>> adj_rev;

vector<bool> winning;
vector<bool> losing;
vector<bool> visited;
vector<int> degree;

void dfs(int v) {
    visited[v] = true;
    for (int u : adj_rev[v]) {
        if (!visited[u]) {
            if (losing[v])
                winning[u] = true;
            else if (--degree[u] == 0)
                losing[u] = true;
            else
                continue;
            dfs(u);
        }
    }
}

for (int i = 0; i < n; i++) {
    if ((winning[i] || losing[i]) && !visited[i])
        dfs(i);
}
```

hopcroft-karp.h

Description: Fast bipartite matching algorithm. Graph g should be a list of neighbors of the left partition, and $btoa$ should be a vector full of -1's of the same size as the right partition. Returns the size of the matching. $btoa[i]$ will be the match for vertex i on the right side, or -1 if it's not matched.

Usage: `\{\}texttt{vector<int> btoa(m, -1); hopcroftKarp(g, btoa);}`

Time: $O(\sqrt{VE})$

```
struct Hop{
    using vi = vector<int>;

    int n, m;
    vector<vi> g;
    vi btoa;

    Hop(int n, int m) : n(n), m(m), g(n+1), btoa(m+1, -1) {}

    void add_edge(int a, int b){
        g[a].push_back(b);
    }

    bool dfs(int a, int L, vi &A, vi &B) { //start-hash
        if (A[a] != L) return 0;
        A[a] = -1;
        for(auto &b : g[a]) if (B[b] == L + 1) {
            B[b] = 0;
            if (btoa[b] == -1 || dfs(btoa[b], L+1, A, B))
                return btoa[b] = a, 1;
        }
        return 0;
    } //end-hash

    int solve() { //start-hash
        int res = 0;
        vector<int> A(g.size()), B(int(btoa.size())), cur,
        ↳ next;
        for (;;) {
            fill(A.begin(), A.end(), 0), fill(B.begin(),
            ↳ B.end(), 0);
            cur.clear();

```

```
for(auto &a : btoa) if (a != -1) A[a] = -1;
for (int a = 0; a < g.size(); ++a) if (A[a] == 0)
    ↳ cur.push_back(a);
for (int lay = 1;; ++lay) {
    bool islast = 0; next.clear();
    for(auto &a : cur) for(auto &b : g[a]) {
        if (btoa[b] == -1) B[b] = lay, islast = 1;
        else if (btoa[b] != a && !B[b])
            B[b] = lay, next.push_back(btoa[b]);
    }
    if (islast) break;
    if (next.empty()) return res;
    for(auto &a : next) A[a] = lay;
    cur.swap(next);
}
for(int a = 0; a < int(g.size()); ++a)
    res += dfs(a, 0, A, B);
} //end-hash
};
```

Hungarian-matching.h

Description: Given a weighted bipartite graph, matches every node on the left with a node on the right such that no nodes are in two matchings and the sum of the edge weights is minimal. Takes $cost[N][M]$, where $cost[i][j]$ = cost for $L[i]$ to be matched with $R[j]$ and returns (min cost, match), where $L[i]$ is matched with $R[match[i]]$. Negate costs for max cost.

Time: $O(N^2M)$

```
template<class cost_t> pair<cost_t, vector<int>>
↳ hungarian(const vector<vector<cost_t>> &a){
    int n = a.size() + 1, m = a[0].size() + 1;

    cost_t INF = 1e9; //Change here

    vector<int> p(m), ans(n - 1);
    vector<cost_t> u(n), v(m);
    for(int i = 1; i < n; ++i) {
        p[0] = i; int j0 = 0;
        vector<cost_t> dist(m, INF);
        vector<int> pre(m, -1);
        vector<bool> done(m + 1);
        do {
            done[j0] = true;
            int i0 = p[j0], j1;
            cost_t delta = INF;
            for(int j = 1; j < m; ++j) if (!done[j]) {
                auto cur = a[i0-1][j-1] - u[i0] - v[j];
                if (cur < dist[j]) dist[j] = cur, pre[j] = j0;
                if (dist[j] < delta) delta = dist[j], j1 = j;
            }
            for(int j = 0; j < m; ++j)
                if (done[j]) u[p[j]] += delta, v[j] -= delta;
            else dist[j] -= delta;
            j0 = j1;
        } while (p[j0]);
        while (j0) {
            int j1 = pre[j0]; p[j0] = p[j1], j0 = j1;
        }
        for(int j = 1; j < m; ++j) if (p[j]) ans[p[j]-1] = j-1;
        return {-v[0], ans};
    }
```

Kosaraju.h

Description: Kosaraju para encontrar componentes fortemente conexas (SCC). Retorna em `\{\}\texttt{scc}` as componentes em ordem topológica.
Time: $O(n + m)$

```
struct Kosa{
    int n, cont;
    vector<int> marc, ord, comp;
    vector<vector<int>> grafo, rgrafo, scc;

    Kosa(int n) : n(n), marc(n+1), grafo(n+1), rgrafo(n+1),
        ↪ comp(n+1), scc(n+1) {}

    void add_edge(int a, int b){
        grafo[a].push_back(b);
        rgrafo[b].push_back(a);
    }

    void dfs1(int v){
        marc[v] = 1;
        for(auto viz : grafo[v]){
            if(!marc[viz]) dfs1(viz);
        }
        ord.push_back(v);
    }

    void dfs2(int v, int c){
        comp[v] = c;
        for(auto viz : rgrafo[v]){
            if(!comp[viz]) dfs2(viz, c);
        }
    }

    void build(){
        cont = 0;

        for(int i = 1; i <= n; i++){
            if(!marc[i]) dfs1(i);
        }

        reverse(ord.begin(), ord.end());

        for(int v : ord){
            if(!comp[v]){
                dfs2(v, ++cont);
            }
        }

        for(int i = 1; i <= n; i++){
            for(int j : grafo[i]){
                if(comp[i] == comp[j]) continue;
                scc[comp[i]].push_back(comp[j]);
            }
        }
    }
};
```

Kuhn.h

Description: Algoritmo de Kuhn para matching bipartido máximo.
Time: $O(VE)$

```
struct bm_t
{
```

```
int N, M, T;
vector<vector<int>> grafo;
vector<int> match, seen;
bm_t(int a, int b) : N(a), M(a+b), T(0),
    ↪ grafo(M), match(M, -1), seen(M, -1) {}

void add_edge(int a, int b){
    grafo[a].push_back(b + N);
}

bool dfs(int cur){
    if(seen[cur] == T) return false;
    seen[cur] = T;
    for(int nxt : grafo[cur]) if(match[nxt] == -1){
        match[nxt] = cur;
        match[cur] = nxt;
        return true;
    }
    for(int nxt : grafo[cur]) if(dfs(match[nxt])){
        match[nxt] = cur;
        match[cur] = nxt;
        return true;
    }
    return false;
}

int solve(){
    int res = 0;
    for(int cur = 1; cur;){
        cur = 0; ++T;
        for(int i = 0; i < N; ++i) if(match[i] == -1)
            cur += dfs(i);
        res += cur;
    }
    return res;
}

};
```

mincostMaxFlow.h

Description: Min-cost max-flow. Assume que não há ciclos negativos.
Usage: `\{\}\texttt{run}(s, t, limFlow)` retorna `\{\}\texttt{(flow, cost)}`.
Time: $O(F(V + E) \log V)$, sendo F a quantidade de fluxo

```
template<class flow_t, class cost_t> struct min_cost {
    static constexpr flow_t FLOW_EPS = flow_t(1e-10);
    static constexpr flow_t FLOW_INF =
        ↪ numeric_limits<flow_t>::max();
    static constexpr cost_t COST_EPS = cost_t(1e-10);
    static constexpr cost_t COST_INF =
        ↪ numeric_limits<cost_t>::max();
    int n, m{}; vector<int> ptr, nxt, zu;
    vector<flow_t> capa; vector<cost_t> cost;

    min_cost(int N) : n(N), ptr(n, -1), dist(n), vis(n), pari(n) {}

    void add_edge(int u, int v, flow_t w, cost_t c) {
        nxt.push_back(ptr[u]); zu.push_back(v);
        ↪ capa.push_back(w);
        cost.push_back(c); ptr[u] = m++;
        nxt.push_back(ptr[v]); zu.push_back(u);
        ↪ capa.push_back(0);
        cost.push_back(-c); ptr[v] = m++;
    }

    vector<cost_t> pot, dist; vector<bool> vis; vector<int>
    ↪ pari;
```

```
vector<flow_t> flows; vector<cost_t> slopes;
// You can pass t = -1 to find a shortestest
void shortestest(int s, int t) { // path to each vertex . //
    ↪ hash=1
    using E = pair<cost_t, int>;
    priority_queue<E, vector<E>, greater<E>> que;
    for(int u = 0; u < n; ++u){dist[u]=COST_INF;
        ↪ vis[u]=false;}
    for (que.emplace(dist[s] = 0, s); !que.empty(); ) {
        const cost_t c = que.top().first;
        const int u = que.top().second; que.pop();
        if (vis[u]) continue;
        vis[u] = true; if (u == t) return;
        for (int i = ptr[u]; ~i; i = nxt[i]) if (capa[i]
            ↪ > FLOW_EPS) {
            const int v = zu[i];
            const cost_t cc = c + cost[i] + pot[u] -
                ↪ pot[v];
            if(dist[v] >
                ↪ cc){que.emplace(dist[v]=cc,v);pari[v]=i;}
        }
    }
    // hash=1 = 89f16a
    auto run(int s, int t, flow_t limFlow = FLOW_INF) { //
        ↪ hash=2
        pot.assign(n, 0); flows = {0}; slopes.clear();
        while (true) {
            bool upd = false;
            for (int i = 0; i < m; ++i) if (capa[i] >
                ↪ FLOW_EPS) {
                const int u = zu[i ^ 1], v = zu[i];
                const cost_t cc = pot[u] + cost[i];
                if(pot[v] > cc + COST_EPS) { pot[v] = cc; upd
                    ↪ = true; }
            } if (!upd) break;
        }
        flow_t flow = 0; cost_t tot_cost = 0;
        while (flow < limFlow) {
            shortestest(s, t); flow_t f = limFlow - flow;
            if (!vis[t]) break;
            for(int u = 0; u < n; ++u) pot[u] +=
                ↪ min(dist[u], dist[t]);
            for (int v = t; v != s; ) { const int i = pari[v];
                if (f > capa[i]) { f = capa[i]; } v = zu[i ^ 1];
            }
            for (int v = t; v != s; ) { const int i = pari[v];
                capa[i] -= f; capa[i ^ 1] += f; v = zu[i ^ 1];
            }
            flow += f; tot_cost += f * (pot[t] - pot[s]);
            flows.push_back(flow); slopes.push_back(pot[t] -
                ↪ pot[s]);
        } return make_pair(flow, tot_cost);
    } // hash=2 = 285527
};
```

TopoSort.h

Description: Algoritmo para encontrar a ordenação topológica de um DAG.
Usage: Retorna um vetor com os vértices em ordem topológica.
Time: $O(N + M)$

```
struct TopoSort
{
    int n;
    vector<int> grau;
    vector<vector<int>> grafo;
```

```

TopoSort(int n): n(n), grau(n+1), grafo(n+1){}

void add_edge(int a, int b){
    grau[b]++;
    grafo[a].push_back(b);
}

vector<int> top_sort(){
    vector<int> resp;
    queue<int> fila;
    for(int i=1; i<=n;i++){
        if(!grau[i])fila.push(i);
    }
    while (!fila.empty())
    {
        int u = fila.front();
        resp.push_back(u);
        fila.pop();
        for(int viz : grafo[u]){
            grau[viz]--;
            if(!grau[viz])fila.push(viz);
        }
    }
    if(resp.size() < n){
        return {};
    }else{
        return resp;
    }
}
};

```

5 Arvore

Centroid.h

Description: Centroid Decomposition. Decompõe a árvore em centróides recursivamente. Constrói a árvore de centróides em $O(n \log n)$.

Usage: `\{\}\texttt{build}(x, p)` constrói a decomposição. A árvore de centróides fica armazenada em `\{\}\texttt{pai}`.

Time: $O(n \log n)$

```

struct Centroid{
    int n;
    vector<int> used, pai, sub;
    vector<vector<int>> vec;

    Centroid(int n) : n(n), used(n+1), pai(n+1), sub(n+1),
        ↪ vec(n+1) {}

    void add_edge(int v, int u){
        vec[v].push_back(u);
        vec[u].push_back(v);
    }

    int dfs_sz(int x, int p=0){
        sub[x]=1;
        for(int i:vec[x]){
            if(i==p || used[i]) continue;
            sub[x]+=dfs_sz(i, x);
        }
        return sub[x];
    }

    int find_c(int x, int total, int p=0){
        for(int i:vec[x]){
            if(i==p || used[i]) continue;
            if(2*sub[i]>total) return find_c(i, total, x);
        }
    }
};

```

```

    }
    return x;
}

void build(int x=1, int p=0){
    int c=find_c(x, dfs_sz(x));

    //do something

    used[c]=1;
    pai[c]=p;
    for(int i:vec[c]){
        if(!used[i]) build(i, c);
    }
}

};

```

DominatorTree.h

Description: Dominator Tree. Computa a árvore de dominadores dos vértices alcançáveis a partir da raiz.

Usage: `\{\}\texttt{build}(r, n)` constrói a árvore de dominadores a partir da raiz `\{\}\texttt{t[r]}`; `\{\}\texttt{t[i]}` armazena os vértices da dominator tree.

Time: $O((V + E) \log V)$

```

const int N = 2e5 + 9;

vector<int> g[N];
vector<int> t[N], rg[N], bucket[N]; //t = dominator tree of
    ↪ the nodes reachable from root
int sdом[N], par[N], idom[N], dsu[N], label[N];
int id[N], rev[N], T;
int find_(int u, int x = 0) {
    if(u == dsu[u]) return x ? -1 : u;
    int v = find_(dsu[u], x+1);
    if(v < 0) return u;
    if(sdom[label[dsu[u]]] < sdom[label[u]]) label[u] =
        ↪ label[dsu[u]];
    dsu[u] = v;
    return x ? v : label[u];
}

void dfs(int u) {
    T++; id[u] = T;
    rev[T] = u; label[T] = T;
    sdом[T] = T; dsu[T] = T;
    for(int i = 0; i < g[u].size(); i++) {
        int w = g[u][i];
        if(!id[w]) dfs(w, par[id[w]] = id[u];
        rg[id[w]].push_back(id[u]);
    }
}

void build(int r, int n) {
    dfs(r);
    n = T;
    for(int i = n; i >= 1; i--) {
        for(int j = 0; j < rg[i].size(); j++) sdом[i] =
            ↪ min(sdom[i], sdом[find_(rg[i][j])]);
        if(i > 1) bucket[sdom[i]].push_back(i);
        for(int j = 0; j < bucket[i].size(); j++) {
            int w = bucket[i][j];
            int v = find_(w);
            if(sdom[v] == sdом[w]) idom[w] = sdом[w];
            else idom[w] = v;
        }
        if(i > 1) dsu[i] = par[i];
    }
    for(int i = 2; i <= n; i++) {

```

```

        if(idom[i] != sdом[i]) idom[i]=idom[idom[i]];
        t[rev[i]].push_back(rev[idom[i]]);
        t[rev[idom[i]]].push_back(rev[i]);
    }
}

int st[N], en[N];
void yo(int u, int pre = 0) {
    st[u] = ++T;
    for(auto v: t[u]) {
        if(v == pre) continue;
        yo(v, u);
    }
    en[u] = T;
}

```

HLD.h

Description: Heavy-Light Decomposition. Decompõe a árvore em caminhos pesados, permitindo queries e updates em caminhos e subárvores com uma SegTree. Suporta valores nas arestas `\{\}\texttt{VALS}\{\}_IN\{\}_EDGES = true` ou nos vértices.

Time: qualquer caminho da árvore é particionado em $O(\log N)$ segmentos; operações em caminho em $O(\log^2 N)$.

```

#include "../data-structures/1D Range Queries (9.2)/LazySeg
    ↪ (15.2).h"

template<int SZ, bool VALS_IN_EDGES> struct HLD {
    int N; vi adj[SZ];
    int par[SZ], root[SZ], depth[SZ], sz[SZ], ti;
    int pos[SZ]; vi rpos; // rpos not used but could be useful
    void ae(int x, int y) { adj[x].pb(y), adj[y].pb(x); }
    void dfsSz(int x) {
        sz[x] = 1;
        each(y,adj[x]) {
            par[y] = x; depth[y] = depth[x]+1;
            adj[y].erase(find(all(adj[y]),x)); /// remove parent
            ↪ from adj list
            dfsSz(y); sz[x] += sz[y];
            if (sz[y] > sz[adj[x][0]]) swap(y,adj[x][0]);
        }
    }
    void dfsHld(int x) {
        pos[x] = ti++; rpos.pb(x);
        each(y,adj[x]) {
            root[y] = (y == adj[x][0] ? root[x] : y);
            dfsHld(y); }
    }
    void init(int _N, int R = 0) { N = _N;
        par[R] = depth[R] = ti = 0; dfsSz(R);
        root[R] = R; dfsHld(R); }
    int lca(int x, int y) {
        for (; root[x] != root[y]; y = par[root[y]])
            if (depth[root[x]] > depth[root[y]]) swap(x,y);
        return depth[x] < depth[y] ? x : y;
    }
    // int dist(int x, int y) { // # edges on path
    // return depth[x]+depth[y]-2*depth[lca(x,y)]; }
    LazySeg<ll,SZ> tree; // segtree for sum
    template <class BinaryOp>
    void processPath(int x, int y, BinaryOp op) {
        for (; root[x] != root[y]; y = par[root[y]]) {
            if (depth[root[x]] > depth[root[y]]) swap(x,y);
            op(pos[root[y]],pos[y]); }
    }
};

```

```

    if (depth[x] > depth[y]) swap(x,y);
    op(pos[x]+VALS_IN_EDGES,pos[y]);
}
void modifyPath(int x, int y, int v) {
    processPath(x,y,[this,&v](int l, int r) {
        tree.upd(l,r,v); }); }
ll queryPath(int x, int y) {
    ll res = 0; processPath(x,y,[this,&res](int l, int r) {
        res += tree.query(l,r); });
    return res; }
void modifySubtree(int x, int v) {
    tree.upd(pos[x]+VALS_IN_EDGES,pos[x]+sz[x]-1,v); }
};

```

Isomorfismo.h

Description: Isomorfismo de árvores. `thash()` retorna o hash da árvore usando centróides como vértices especiais. Duas árvores são isomorfas se e somente se tem hashes iguais.

Time: $O(|V|\log|V|)$

```
map<vector<int>, int> mphash;
```

```

struct tree {
    int n;
    vector<vector<int>> g;
    vector<int> sz, cs;

    tree(int n_) : n(n_), g(n_), sz(n_) {}

    void dfs_centroid(int v, int p) {
        sz[v] = 1;
        bool cent = true;
        for (int u : g[v]) if (u != p) {
            dfs_centroid(u, v), sz[v] += sz[u];
            if (sz[u] > n/2) cent = false;
        }
        if (cent and n - sz[v] <= n/2) cs.push_back(v);
    }
    int fhash(int v, int p) {
        vector<int> h;
        for (int u : g[v]) if (u != p) h.push_back(fhash(u, v));
        sort(h.begin(), h.end());
        if (!mphash.count(h)) mphash[h] = mphash.size();
        return mphash[h];
    }
    ll thash() {
        cs.clear();
        dfs_centroid(0, -1);
        if (cs.size() == 1) return fhash(cs[0], -1);
        ll h1 = fhash(cs[0], cs[1]), h2 = fhash(cs[1], cs[0]);
        return (min(h1, h2) << 30) + max(h1, h2);
    }
};

```

LCA.h

Description: LCA com binary lifting. Encontra o menor ancestral comum, distância entre vértices e verifica se um vértice está no caminho entre dois outros.

Usage: `build(x)` constrói a estrutura a partir da raiz (padrão: 1). Trocar se a raiz não for 1 no construtor e em `build()`.

Time: $O(n\log n)$ de build, $O(\log n)$ por query

```

struct LCA{
    int n;

```

```

    const int sz = 32;
    vector<int> marc, height;
    vector<vector<int>> g, bl;

    //Trocar se a raiz nao for 1
    LCA(int n) : n(n), g(n+1), bl(sz, vector<int> (n+1, 1)),
        ↪ marc(n+1), height(n+1){}

    void add_edge(int a, int b){
        g[a].push_back(b);
        g[b].push_back(a);
    }

    //Trocar se a raiz nao for 1
    void build(int x = 1){
        marc[x] = 1;
        for(int i = 1; i < sz; i++){
            bl[i][x] = bl[i-1][bl[i-1][x]];
        }

        for(auto viz : g[x]){
            if(marc[viz]) continue;
            bl[0][viz] = x;
            height[viz] = height[x]+1;
            build(viz);
        }

        int find_lca(int a, int b){
            if(height[a] < height[b]) swap(a,b);

            int dif = height[a] - height[b];
            for(int i = 0; i < sz; i++){
                if((1<<i) & dif){
                    a = bl[i][a];
                }
            }

            assert(height[a] == height[b]);
            if(a == b) return a;

            for(int i = sz-1; i >=0; i--){
                if(bl[i][a] == bl[i][b]) continue;
                a = bl[i][a];
                b = bl[i][b];
            }

            assert(a != b);
            assert(bl[0][a] == bl[0][b]);
            return bl[0][a];
        }

        int dist(int a, int b){
            int l = find_lca(a,b);
            return height[a] + height[b] - 2*height[l];
        }

        // Returns true if x is on the simple path between a and b
        bool entre(int x, int a, int b){
            return dist(a, b) == dist(a, x) + dist(x, b);
        }
};

```

```
};
```

LCAQ.h

Description: LCA com binary lifting e query de soma em caminhos. Armazena valores nos vértices e responde queries de soma no caminho entre dois nós.

Usage: `set_val(i, x)` define o valor do vértice `i`. `build(x)` constrói a estrutura a partir da raiz (padrão: 1). Trocar se a raiz não for 1 no construtor e em `build()`. Mudar o valor neutro (`neutral`) e a operação de junção em `tb`.

Time: $O(n\log n)$ de build, $O(\log n)$ por query

```

struct LCAQ{

    const int neutral = 0;
    int n;
    const int sz = 32;
    vector<int> marc, height, val;
    vector<vector<int>> g, bl, tb;

    //Trocar se a raiz nao for 1
    LCAQ(int n) : n(n), g(n+1), bl(sz, vector<int> (n+1, 1)),
        ↪ tb(sz, vector<int> (n+1, 1)), marc(n+1), height(n+1){}

    void add_edge(int a, int b){
        g[a].push_back(b);
        g[b].push_back(a);
    }

    void set_val(int i, int x){
        val[i] = x;
    }

    //Trocar se a raiz nao for 1
    void build(int x = 1){
        marc[x] = 1;
        for(int i = 1; i < sz; i++){
            bl[i][x] = bl[i-1][bl[i-1][x]];
            tb[i][x] = tb[i-1][x] + tb[i-1][bl[i-1][x]];
        }

        for(auto viz : g[x]){
            if(marc[viz]) continue;
            bl[0][viz] = x;
            tb[0][viz] = val[x];
            height[viz] = height[x]+1;
            build(viz);
        }

        int find_lca(int a, int b){
            if(height[a] < height[b]) swap(a,b);

            int dif = height[a] - height[b];
            for(int i = 0; i < sz; i++){
                if((1<<i) & dif){
                    a = bl[i][a];
                }
            }

            assert(height[a] == height[b]);
            if(a == b) return a;

            for(int i = sz-1; i >=0; i--){
                if(bl[i][a] == bl[i][b]) continue;
                a = bl[i][a];
                b = bl[i][b];
            }

            assert(a != b);
            assert(bl[0][a] == bl[0][b]);
            return bl[0][a];
        }
};

```

```

int dist(int a, int b){
    int l = find_lca(a,b);
    return height[a] + height[b] - 2*height[l];
}

int query_lca(int a, int lca){
    int ans = neutral;
    int dif = height[lca] - height[a];
    for(int i = 0; i < sz; i++){
        if((1<<i) & dif){
            ans = ans + tb[i][a];
            a = bl[i][a];
        }
    }
    assert(a == lca);
    return ans;
};

int query(int a, int b){
    int lca = find_lca(a,b);
    return val[a] + val[b] + query(a,lca) + query(b,lca)
    ↪ - val[lca];
}
};

```

VitualTree.h

Description: Virtual Tree. Dado um conjunto de vértices, constrói uma árvore minimal contendo esse conjunto. A virtual tree possui no máximo $2|V| - 1$ vértices.

Usage: `\{\}texttt{virt[i]}` guarda os vizinhos do vértice `\{\}texttt{i}` (pares: vértice, distância). `\{\}texttt{build(v)}` retorna a raiz da virtual tree.

Time: $O(N \log N)$, sendo N o tamanho do conjunto de vértices passados.

```
vector<pair<int, int>> virt[mxn];
```

```

int build(vector<int> v){
    auto cmp = [&](int a, int b){return tempo[a]<tempo[b];};
    sort(v.begin(), v.end(), cmp);
    for(int i=(int)v.size()-1; i>0; i--){
        ↪ v.push_back(lca(v[i], v[i-1]));
    }
    sort(v.begin(), v.end(), cmp);
    v.erase(unique(v.begin(), v.end()), v.end());
    for(int i=0; i<v.size(); i++) virt[v[i]].clear();
    for(int i=1; i<v.size(); i++) virt[lca(v[i],
        ↪ v[i-1])].clear();
    for(int i=1; i<v.size(); i++){
        int pai = lca(v[i], v[i-1]);
        int d = dist(pai, v[i]);
        virt[pai].emplace_back(v[i], d);
    }
    return v[0];
}

```

6 DataStructures

BIT.h

Description: Fenwick Tree (BIT) para range sum com update pontual. Suporta função associativa com inversos em um conjunto com elemento neutro. Para máximo: apenas update de incremento (só é possível aumentar valores).

Time: $O(\log n)$ por query/update

```

struct FenwickTree {
    vector<int> bit;
    int n;

    FenwickTree(int n) {
        this->n = n;
        bit.assign(n, 0);
    }

    FenwickTree(vector<int> const &a) : FenwickTree(a.size()){
        for (int i = 0; i < n; i++) {
            bit[i] += a[i];
            int r = i | (i + 1);
            if (r < n) bit[r] += bit[i];
        }
    }

    int sum(int r) {
        int ret = 0;
        for (; r >= 0; r = (r & (r + 1)) - 1)
            ret += bit[r]; //ret = max(ret, bit[r]);
        return ret;
    }

    int sum(int l, int r) {
        return sum(r) - sum(l - 1);
    }

    void add(int idx, int delta) {
        for (; idx < n; idx = idx | (idx + 1))
            bit[idx] += delta; //bit[idx] = max(bit[idx],
            ↪ delta);
    }
};

```

BIT2D.h

Description: BIT 2D de soma para queries offline/sparse. Suporta update pontual e range sum.

Usage: É necessário construir um vetor de pontos que serão atualizados.

Time: $O(n \log n)$ build, $O(\log^2 n)$ update/query

```

template<class T = int> struct bit2d {
    vector<T> X;
    vector<vector<T>> Y, t;

    int ub(vector<T>& v, T x) {
        return upper_bound(v.begin(), v.end(), x) - v.begin();
    }

    bit2d(vector<pair<T, T>> v) {
        for (auto [x, y] : v) X.push_back(x);
        sort(X.begin(), X.end());
        X.erase(unique(X.begin(), X.end()), X.end());

        t.resize(X.size() + 1);
        Y.resize(t.size());
        sort(v.begin(), v.end(), [](auto a, auto b) {
            return a.second < b.second; });
        for (auto [x, y] : v) for (int i = ub(X, x); i <
            ↪ t.size(); i += i&-i)
            if (!Y[i].size() or Y[i].back() != y) Y[i].push_back(y);

        for (int i = 0; i < t.size(); i++)
            ↪ t[i].resize(Y[i].size() + 1);
    }

    //Adds v to [x,y]
    void update(T x, T y, T v) {

```

```

        for (int i = ub(X, x); i < t.size(); i += i&-i)
            for (int j = ub(Y[i], y); j < t[i].size(); j += j&-j)
                ↪ t[i][j] += v;
    }

    T query(T x, T y) {
        T ans = 0;
        for (int i = ub(X, x); i; i -= i&-i)
            for (int j = ub(Y[i], y); j; j -= j&-j) ans += t[i][j];
        return ans;
    }

    T query(T x1, T y1, T x2, T y2) {
        return query(x2, y2)-query(x2, y1-1)-query(x1-1,
            ↪ y2)+query(x1-1, y1-1);
    }
};

```

BitRangeUpdate.h

Description: BIT com range update e range query de soma. Usa duas BITs para implementar range add e range sum.

Time: $O(\log n)$ por query/update

```

vector<int> bit1, bit2;
void init(int n){
    bit1.assign(n+1, 0);
    bit2.assign(n+1, 0);
}

int rsq(vector<int> &bit, int i){
    int ans = 0;
    for(; i; i-=i&-i)
        ans += bit[i];
    return ans;
}

void update(vector<int> &bit, int i, int v){
    for(; i < bit.size(); i+=i&-i)
        bit[i] += v;
}

void update(int i, int j, int v){
    update(bit1, i, v);
    update(bit1, j+1, -v);
    update(bit2, i, v*(i-1));
    update(bit2, j+1, -v*j);
}

int rsq(int i){
    return rsq(bit1, i)*i - rsq(bit2, i);
}

int rsq(int i, int j){
    return rsq(j) - rsq(i-1);
}

```

Dsu.h

Description: Disjoint Set Union (DSU) com path compression e union by rank.

Time: $O(\alpha(n))$ amortizado por operação

```

struct DSU
{
    int n;
    vector<int> pai, rank;

    DSU(int n) : n(n), pai(n+1), rank(n+1,1){
        for(int i = 1; i <= n; i++){
            pai[i] = i;
        }

        int find(int a){
            if(pai[a] == a) return a;
            return pai[a] = find(pai[a]);
        }

        void uu(int a, int b){
            a = find(a), b = find(b);
            if(a == b) return;
            if(rank[a] > rank[b]) swap(a,b);
            rank[b] += rank[a];
            pai[a] = b;
        }
    };
};

```

LazySegTree.h

Description: Segment Tree com Lazy Propagation para range add e range sum.

Usage: `\{\}texttt{update(l, r, val)}` adiciona `\{\}texttt{val}` ao range `[l, r]`.

Time: $O(\log n)$ por query/update

```

struct SegTree{
    int n;
    struct Node{
        int val;
        Node operator+(Node other) const{
            return {this->val + other.val};
        }
        Node operator=(int x){
            return {this->val = x};
        }
    };
    Node neutral = {0};
    vector<Node> t;
    vector<int> lazy; //Change variable

    SegTree(vector<int> a){
        n = a.size();
        t.resize(4*n);
        lazy.resize(4*n); //Neutral element
        build(a, 1, 0, n-1);
    }

    void build(vector<int>& a, int v, int tl, int tr) {
        if (tl == tr) {
            t[v] = a[tl];
        } else {
            int tm = (tl + tr) / 2;
            build(a, v*2, tl, tm);
            build(a, v*2+1, tm+1, tr);
            t[v] = t[v*2] + t[v*2+1];
        }
    }

    void unlazy(int v, int tl, int tr){
        // if(lazy[v] == 0) return; //Neutral element

```

```

//Update current range
t[v].val += (tr-tl+1) * lazy[v];

if(tl != tr){
    lazy[2*v] += lazy[v];
    lazy[2*v+1] += lazy[v];
}

lazy[v] = 0;
}

Node query(int l, int r){
    return query(1, 0, n-1, l, r);
}

Node query(int v, int tl, int tr, int l, int r){
    unlazy(v, tl, tr);
    if (l > r) {
        return neutral;
    }
    if (l == tl && r == tr) {
        return t[v];
    }
    int tm = (tl + tr) / 2;
    return query(v*2, tl, tm, l, min(r, tm))
        + query(v*2+1, tm+1, tr, max(l, tm+1), r);
}

void update(int pos, int val){
    update(1, 0, n-1, pos, pos, val);
}

void update(int l, int r, int val){
    update(1, 0, n-1, l, r, val);
}

void update(int v, int tl, int tr, int l, int r, int val){
    unlazy(v, tl, tr);
    if (l > r) {
        return;
    }
    if (l == tl && r == tr) {
        lazy[v] += val; //Change here
        unlazy(v, tl, tr);
        return;
    }
    int tm = (tl + tr) / 2;
    update(v*2, tl, tm, l, min(r, tm), val);
    update(v*2+1, tm+1, tr, max(l, tm+1), r, val);
    t[v] = t[v*2] + t[v*2+1];
}
};

```

lineContainer.h

Description: Container onde é possível adicionar retas da forma $kx + m$ e consultar o valor máximo em pontos x . Útil para DP (convex hull trick).

Time: $O(\log n)$ por operação

```

struct Line {
    mutable ll k, m, p;
    bool operator<(const Line& o) const { return k < o.k; }
    bool operator<(ll x) const { return p < x; }
};

struct LineContainer : multiset<Line, less<>> {
    static const ll inf = LLONG_MAX; //for doubles 1/0

```

```

    ll div(ll a, ll b) { //for doubles return a/b
        return a / b - ((a ^ b) < 0 && a % b); }

    bool isect(iterator x, iterator y) {
        if (y == end()) { x->p = inf; return false; }
        if (x->k == y->k) x->p = x->m > y->m ? inf : -inf;
        else x->p = div(y->m - x->m, x->k - y->k);
        return x->p >= y->p;
    }

    //para achar o mínimo, é preciso fazer insert({-k, -m,
    ↪ 0}), além disso multiplicar por -1 o resultado da
    ↪ query
    void add(ll k, ll m) {
        auto z = insert({k, m, 0}), y = z++, x = y;
        while (isect(y, z)) z = erase(z);
        if (x != begin() && isect(--x, y)) isect(x, y = erase(y));
        while ((y = x) != begin() && (--x)->p >= y->p)
            isect(x, erase(y));
    }

    ll query(ll x) {
        assert(!empty());
        auto l = *lower_bound(x);
        return l.k * x + l.m;
    }
};

```

MergeSortTree.h

Description: Nó de SegTree para Merge Sort Tree. Cada nó armazena um vetor ordenado. O operador `\{\}texttt{+}` faz merge de dois vetores ordenados.

Time: $O(n \log n)$ build, $O(\log^2 n)$ por query

```

//Segtree node for Merge-Sort
struct Node{
    vector<int> vec;
    Node operator+(Node other) const{
        vector<int> novo(vec.size() + other.vec.size());
        merge(vec.begin(), vec.end(), other.vec.begin(),
            ↪ other.vec.end(), novo.begin());
        return {novo};
    }
    Node operator=(int x){
        return {this->vec = {x}};
    }
};

```

Mo.h

Description: Template do Algoritmo de Mo. Ordena queries em blocos de tamanho $\sqrt{N}$ para responder range queries offline.

Time: $O((N + Q)\sqrt{N})$

```

const int blockSize = 500;

struct Query
{
    int l, r, idx;

    bool operator<(Query other) const{
        return make_pair(1/blockSize, r) <
            ↪ make_pair(other.l/blockSize, other.r);
    };
};

```

```

};

struct Mo{
    //TODO: declare the data structures
    Mo(){

    }

    void add(int idx){
        //TODO: add an element to the data structure
    }

    void remove(int idx){
        //TODO: remove an element from the data structure
    }

    int get_answer(){
        //TODO: get answer from the data structure
    }

    vector<int> solve(vector<Query> queries) {
        vector<int> answers(queries.size());
        sort(queries.begin(), queries.end());

        //TODO: initialize data structure

        int cur_l = 0;
        int cur_r = -1;
        for (Query q : queries) {
            while (cur_l > q.l) {
                cur_l--;
                add(cur_l);
            }
            while (cur_r < q.r) {
                cur_r++;
                add(cur_r);
            }
            while (cur_l < q.l) {
                remove(cur_l);
                cur_l++;
            }
            while (cur_r > q.r) {
                remove(cur_r);
                cur_r--;
            }
            answers[q.idx] = get_answer();
        }
        return answers;
    }
};

```

Pref2D.h

Description: Prefix Sum 2D. Precomputa soma de prefixos para responder range sum queries em $O(1)$.

Usage: `query(rowl, rowr, coll, colr)` retorna a soma no retângulo.

Time: $O(1)$ por query, $O(nm)$ build

```

struct pref2D{
    int n, m;
    vector<vector<int>> mat, pref;

    pref2D(int n, int m, vector<vector<int>> tmp){
        this->n = n; this->m = m;
        mat = tmp;

        pref.resize(n+1);
        for(auto& v : pref) v.resize(m+1, 0);

        for(int i = 1; i <= n; i++){

```

```

            for(int j = 1; j <= m; j++){
                pref[i][j] = pref[i-1][j] + pref[i][j-1] -
                    pref[i-1][j-1] + mat[i-1][j-1];
            }
        }

        int query(int rowl, int rowr, int coll, int colr){
            rowl++, rowr++, coll++, colr++;
            if(rowl > rowr) swap(rowl, rowr);
            if(coll > colr) swap(colr, coll);
            return pref[rowr][colr] - pref[rowl-1][colr] -
                pref[rowr][coll-1] + pref[rowl-1][coll-1];
        }
    };

```

SegTree.h

Description: Segment Tree genérica com operação associativa. Definir struct Node com `operator+` e `operator=`.

Time: $O(\log n)$ por query/update

```

struct SegTree{
    int n;
    struct Node{
        int val;
        Node operator+(Node other) const{
            return {this->val + other.val};
        }
        Node operator=(int x){
            return {this->val = x};
        }
    };
    Node neutral = {0};
    vector<Node> t;

    SegTree(vector<int> a){
        n = a.size();
        t.resize(4*n);
        build(a, 1, 0, n-1);
    }

    void build(vector<int>& a, int v, int tl, int tr) {
        if (tl == tr) {
            t[v] = a[tl];
        } else {
            int tm = (tl + tr) / 2;
            build(a, v*2, tl, tm);
            build(a, v*2+1, tm+1, tr);
            t[v] = t[v*2] + t[v*2+1];
        }
    }

    Node query(int l, int r){
        return query(1, 0, n-1, l, r);
    }

    Node query(int v, int tl, int tr, int l, int r){
        if (l > r)
            return neutral;
        if (l == tl && r == tr) {
            return t[v];
        }
        int tm = (tl + tr) / 2;
        return query(v*2, tl, tm, l, min(r, tm))
            + query(v*2+1, tm+1, tr, max(l, tm+1), r);
    }

    void update(int pos, int val){

```

```

        update(1, 0, n-1, pos, val);
    }

    void update(int v, int tl, int tr, int pos, int new_val){
        if (tl == tr) {
            t[v] = new_val;
        } else {
            int tm = (tl + tr) / 2;
            if (pos <= tm)
                update(v*2, tl, tm, pos, new_val);
            else
                update(v*2+1, tm+1, tr, pos, new_val);
            t[v] = t[v*2] + t[v*2+1];
        }
    }
};

```

SparseSegLazy.h

Description: Sparse Segment Tree com Lazy Propagation. Suporta range add e query de min/max. `add(ql, qr, v)` adiciona v ao range $[ql, qr]$. `get()` retorna `{min, max}` no range.

Time: $O(\log N)$ por query/update

```

struct SparseSegTree {
    struct Node {
        ll mn, mx, lazy;
        int l, r;
        Node() : mn(0), mx(0), lazy(0), l(-1), r(-1) {}
    };

    vector<Node> tree;
    ll L_BOUND, R_BOUND;

    SparseSegTree(ll L, ll R) {
        L_BOUND = L;
        R_BOUND = R;
        tree.reserve(15000000);
        tree.emplace_back();
    }

    void push(int node) {
        if (tree[node].lazy != 0) {
            ll lz = tree[node].lazy;

            if (tree[node].l == -1) {
                tree[node].l = tree.size();
                tree.emplace_back();
            }
            if (tree[node].r == -1) {
                tree[node].r = tree.size();
                tree.emplace_back();
            }

            int l = tree[node].l;
            int r = tree[node].r;

            tree[l].mn += lz;
            tree[l].mx += lz;
            tree[l].lazy += lz;

            tree[r].mn += lz;
            tree[r].mx += lz;
            tree[r].lazy += lz;

            tree[node].lazy = 0;
        }
    }

```



```

}

void update(int node, ll tl, ll tr, ll ql, ll qr, ll v) {
    if (ql <= tl && tr <= qr) {
        tree[node].mn += v;
        tree[node].mx += v;
        tree[node].lazy += v;
        return;
    }
    push(node);
    ll mid = tl + (tr - tl) / 2;

    if (ql <= mid) {
        if (tree[node].l == -1) {
            tree[node].l = tree.size();
            tree.emplace_back();
        }
        update(tree[node].l, tl, mid, ql, qr, v);
    }
    if (qr > mid) {
        if (tree[node].r == -1) {
            tree[node].r = tree.size();
            tree.emplace_back();
        }
        update(tree[node].r, mid + 1, tr, ql, qr, v);
    }

    ll min_val = 2e18, max_val = -2e18;
    int left_child = tree[node].l;
    int right_child = tree[node].r;

    if (left_child != -1) {
        min_val = min(min_val, tree[left_child].mn);
        max_val = max(max_val, tree[left_child].mx);
    } else {
        min_val = min(min_val, 0LL);
        max_val = max(max_val, 0LL);
    }

    if (right_child != -1) {
        min_val = min(min_val, tree[right_child].mn);
        max_val = max(max_val, tree[right_child].mx);
    } else {
        min_val = min(min_val, 0LL);
        max_val = max(max_val, 0LL);
    }

    tree[node].mn = min_val;
    tree[node].mx = max_val;
}

pair<ll, ll> query(int node, ll tl, ll tr, ll ql, ll qr) {
    if (ql <= tl && tr <= qr) {
        return {tree[node].mn, tree[node].mx};
    }
    push(node);
    ll mid = tl + (tr - tl) / 2;
    pair<ll, ll> res = {2e18, -2e18};

    if (ql <= mid) {
        if (tree[node].l != -1) {
            auto p = query(tree[node].l, tl, mid, ql, qr);
            res.first = min(res.first, p.first);
            res.second = max(res.second, p.second);
        } else {
            res.first = min(res.first, 0LL);
            res.second = max(res.second, 0LL);
        }
    }
    if (qr > mid) {

```

```

        if (tree[node].r != -1) {
            auto p = query(tree[node].r, mid + 1, tr, ql,
                ↪ qr);
            res.first = min(res.first, p.first);
            res.second = max(res.second, p.second);
        } else {
            res.first = min(res.first, 0LL);
            res.second = max(res.second, 0LL);
        }
    }
    return res;
}

void add(ll ql, ll qr, ll v) {
    update(0, L_BOUND, R_BOUND, ql, qr, v);
}

pair<ll, ll> get_minmax(ll ql, ll qr) {
    return query(0, L_BOUND, R_BOUND, ql, qr);
}
};

```

SparseTable.h

Description: Sparse Table para range minimum queries (RMQ).
 $\{\}$ texttt{query(l, r)} retorna o mínimo no range $[l, r]$ em $O(1)$.
 $\{\}$ texttt{querylog(l, r)} decompõe por potências de 2.
Time: $O(n \log n)$ build, $O(1)$ query

```

struct SparseTable{
    int K = 25, n;
    vector<vector<int>> st; //st[i][j] = min on range [j, j +
        ↪ 2^i-1]
    vector<int> lg2; //lg2[i] = floor(log2(i))

    SparseTable(vector<int> arr){
        n = arr.size();
        st.resize(K+1);
        for(auto& v : st) v.resize(n);

        st[0] = arr;
        for(int i = 1; i <= K; i++){
            for(int j = 0; j + (1 << i) - 1 < n; j++){
                st[i][j] = min(st[i-1][j], st[i-1][j + (1 <<
                    ↪ (i - 1))]);
            }
        }

        lg2.resize(n+1);
        lg2[1] = 0;
        for(int i = 2; i <= n; i++){
            lg2[i] = lg2[i/2] + 1;
        }
    }

    int query(int l, int r){
        int i = lg2[r-l+1];
        return min(st[i][l], st[i][r-(1<<i)+1]);
    }

    int querylog(int l, int r){

        int neutral = 1e9; //elemento neutro
        int dif = r-l+1;

        for(int i = 0; i <= K; i++){
            if((1<<i) & dif){
                neutral = min(neutral, st[i][l]);
                l = l + (1<<i);
            }
        }
    }
};

```

```

    }

    return neutral;
}

};

```

SparseSegTree.h

Description: Sparse Segment Tree com alocação dinâmica de nós.
 $\{\}$ texttt{add(k, x)} adiciona x na posição k . $\{\}$ texttt{get\{\}_sum(lq, rq)}
 retorna soma no range $[lq, rq]$.
Time: $O(\log N)$ por query/update

```

struct Node {
    int left, right;
    int sum = 0;
    Node *left_child = nullptr, *right_child = nullptr;

    Node(int lb, int rb) {
        left = lb;
        right = rb;
    }

    void extend() {
        if (!left_child && left + 1 < right) {
            int t = (left + right) / 2;
            left_child = new Node(left, t);
            right_child = new Node(t, right);
        }
    }

    void add(int k, int x) {
        extend();
        sum += x;
        if (left_child) {
            if (k < left_child->right)
                left_child->add(k, x);
            else
                right_child->add(k, x);
        }
    }

    int get_sum(int lq, int rq) {
        if (lq <= left && right <= rq)
            return sum;
        if (max(left, lq) >= min(right, rq))
            return 0;
        extend();
        return left_child->get_sum(lq, rq) +
            ↪ right_child->get_sum(lq, rq);
    }
};

```

7 DP

Bitmask.h

Description: Itera sobre todos os subconjuntos estritos de uma máscara (Submask DP).
Time: $O(3^n)$

```

for (int mask = 0; mask < (1 << n); mask++) {
    for (int submask = mask; submask != 0; submask = (submask
        ↪ - 1) & mask) {

```

```

    int subset = mask ^ submask;
    // do whatever you need to do here
}
}

```

DcDp.h

Description: Divide and Conquer DP. Particiona um array em k subarrays minimizando a soma das queries. Assume que $\text{query}(l, r)$ é $O(1)$.

Usage: $\text{DC}(n, k)$ retorna o custo mínimo de particionar n elementos em k subarrays. O usuário deve implementar $\text{query}(l, r)$ que retorna o custo do subarray $[l, r]$.

Time: $O(kn \log n)$

```

11 dp[MAX][2];

void solve(int k, int l, int r, int lk, int rk) {
    if (l > r) return;
    int m = (l+r)/2, p = -1;
    auto& ans = dp[m][k&1] = LINF;
    for (int i = max(m, lk); i <= rk; i++) {
        ll at = dp[i+1][~k&1] + query(m, i);
        if (at < ans) ans = at, p = i;
    }
    solve(k, l, m-1, lk, p), solve(k, m+1, r, p, rk);
}

11 DC(int n, int k) {
    dp[n][0] = dp[n][1] = 0;
    for (int i = 0; i < n; i++) dp[i][0] = LINF;
    for (int i = 1; i <= k; i++) solve(i, 0, n-i, 0, n-i);
    return dp[0][k&1];
}

```

DigitDp.h

Description: Digit DP template. Exemplo: números sem dois dígitos adjacentes iguais. $\text{func}(a, b)$ retorna a quantidade de números no intervalo $[a, b]$.

Usage: O usuário deve modificar a condição dentro do loop em solve para implementar a restrição desejada sobre os dígitos.

Time: $O(d \cdot 10)$ onde d é o número de dígitos

```

11 solve(string &s, int i, int tight, int last, int started){
    if(i==(int)s.size()) return 1;

    if(!tight && dp[i][last][started]!=-1) return
    ↪ dp[i][last][started];

    int lim=(tight?s[i]-'0':9);

    ll resp=0;
    for(int j=0; j<=lim; j++){
        if(started && j==last) continue;
        resp+=solve(s, i+1, tight&(j==lim), j, (started|j)>0);
    }

    if(!tight) return dp[i][last][started]=resp;
    return resp;
}

11 func(ll a, ll b){
    string agr1=to_string(a-1);
    memset(dp, -1, sizeof(dp));
    ll ans1 = solve(agr1, 0, 1, 10, 0);

    string agr2=to_string(b);

```

```

memset(dp, -1, sizeof(dp));
ll ans2 = solve(agr2, 0, 1, 10, 0);

return ans2-ans1;
}

```

Sos.h

Description: Sum over Subsets (SOS) DP. Dado um array A de 2^N inteiros, calcula $F[\text{mask}] = \sum_{i \subseteq \text{mask}} A[i]$.

Time: $O(N \cdot 2^N)$

```

for(int i = 0; i < (1<<N); ++i)
    F[i] = A[i];
for(int i = 0; i < N; ++i) for(int mask = 0; mask < (1<<N);
    ↪ ++mask){
    if(mask & (1<<i))
        F[mask] += F[mask^(1<<i)];
}

```

SubsetSum.h

Description: Subset Sum com bounded knapsack otimizado por sliding window. Soma desejada = S , números = n . Itens não podem ter valor 0 nem frequência 0.

Usage: sack é um vetor de pares $\{\text{item}, \text{frequência}\}$. $\text{dp}[x] = 1$ se é possível obter soma x , 0 caso contrário.

Time: $O(S\sqrt{n})$ runtime, $O(n)$ memory

```

vector<pair<int,int>> sack; // {item, frequency}
vector<int> dp(S+1, 0);
dp[0] = 1;

for(int i = 0; i < sack.size(); i++){
    vector<int> ndp(S+1);
    auto [item, freq] = sack[i];
    for(int j = 0; j < item; j++){ //starting at position j
        int numTrues = 0;
        for(int k = j; k <= S; k += item){
            ndp[k] = dp[k];
            if(numTrues > 0) ndp[k] = true;
            if(k - freq*item >= 0) numTrues -= dp[k -
            ↪ freq*item];
            numTrues += dp[k];
        }
    }
    swap(ndp, dp);
}

```

8 Extra

8.1 String Matching with Wildcards

Consider a text T and a pattern P . P and T may have wildcards that will match any character. The problem is to get the positions where P occur in T .

If we define the value of the characters such that the wildcard is zero and the other characters are positive, there is a matching at position i iff $\sum_{j=0}^{|P|-1} P[j]T[i+j](P[j]-T[i+j])^2 = 0$. Then, one can evaluate each term of

$$\sum_{j=0}^{|P|-1} (P[j]^3 T[i+j] - 2P[j]^2 T[i+j]^2 + P[j]T[i+j]^3)$$

using three convolutions.

2pointer.h

Description: Técnica dos dois ponteiros (Two-Pointer). Modelos para: (1) Maior segmento bom (segmentos bons são fechados sob aninhamento). (2) Menor segmento bom (segmentos bons são fechados sob contenção). Inclui notas sobre quando aplicar o método.

Time: $O(n)$

```

x = 0, L = 0
for R = 0..n-1
    x += a[R]
    while x > s:
        x -= a[L]
        L++
    res = max(res, R - L + 1)

```

```

x = 0, L = 0
for R = 0..n-1
    x += a[R]
    while x - a[L] >= s:
        x -= a[L]
        L++
    if x >= s:
        res = min(res, R - L + 1)

```

Suppose you received a problem at the contest in which you need to do something similar: find the longest (shortest) good segment or count the number of good segments. How do you know if you can apply the two-pointer method to it? First, one of the following two properties must be met:

```

if the segment [L,R] is good, then any segment nested in
    ↪ it is also good (in this case, you can apply the code
    ↪ from the first problem);
if the segment [L,R] is good, then any segment that
    ↪ contains it is also good (in this case, you can apply
    ↪ the code from the second problem).

```

Secondly, you should be able to recalculate your function (check if current segment is good or bad), while moving the left or right border by one to the right. In such tasks, the code almost always looks like this:

```

L = 0
for R = 0..n-1
    add(a[R])
    while not good():
        remove(a[L])
        L++

```

BitOperators.h

Description: Operações bit a bit built-in, Gray Code e identidades bitwise. $\texttt{_builtin_clz}$, $\texttt{_builtin_ctz}$, $\texttt{_builtin_popcount}$, $\texttt{_builtin_parity}$. Gray code: $g(n) = n \oplus (n \gg 1)$ e sua inversa.

Time: $O(1)$ por operação

Pra fazer funcionar com `long long` é só colocar `ll` no `final` da função, por exemplo `"__builtin_clz(x)"` vira `"__builtin_clzll(x)"`

`__builtin_clz(x)`: Count leading zeros. Retorna o número de zeros no início da representação binária de `x` (incio == bits mais significativos)

`__builtin_ctz(x)`: Count trailing zeros. Retorna o número de zeros no `final` da representação binária de `x`

`__builtin_popcount(x)`: Population count. Retorna o número de bits definidos como `1` na representação binária de `x`

`__builtin_parity(x)`: Parity. Retorna `1` se o número de bits definidos como `1` for ímpar e `0` se for par.

`int L = 32 - __builtin_clz(n), n = 1 << L; : "n" se torna a primeira potencia de 2 estritamente maior que n`

```
// I'ésimo numero do GrayCode
int g (int n) {
    return n ^ (n >> 1);
}
```

```
// GrayCode Reverso
int rev_g (int g) {
    int n = 0;
    for (; g; g >>= 1)
        n ^= g;
    return n;
}
```

Some properties of bitwise operations:

```
a|b = a~b + a&b
a^(a&b) = (a|b)^b
b^(a&b) = (a|b)^a
(a&b)^(a|b) = a^b
```

Addition:

```
a+b = a|b + a&b
a+b = a^b + 2(a&b)
```

Subtraction:

```
a-b = (a^(a&b))-((a|b)^a)
a-b = ((a|b)^b)-((a|b)^a)
a-b = (a^(a&b))-(b^(a&b))
a-b = ((a|b)^b)-((b^(a&b))
```

Brute.h

Description: Script bash para stress testing. Compila code, brute e gen, gera testes com seeds crescentes e compara as saidas com `\{\}\texttt{diff}`. Para no primeiro WA e imprime o caso falho.

Time: N/A

```
set -e
g++ code.cpp -o code
g++ brute.cpp -o brute
g++ gen.cpp -o gen
for((i = 1; ; ++i)) do
    echo "Test: " $i
    ./gen $i > input_file
    cat input_file
    ./code < input_file > myAnswer
    ./brute < input_file > correctAnswer
    diff -Z myAnswer correctAnswer > /dev/null || break
    cat input_file
    cat myAnswer
    cat correctAnswer
    echo "Passed test" $i
done
echo "WA:"
```

```
cat input_file
echo "My:"
cat myAnswer
echo "correct:"
cat correctAnswer
```

CoordinateCompression.h

Description: Compressão de coordenadas. Mapeia valores de um array para inteiros consecutivos $[0, k)$, preservando a ordem relativa.

Usage: Para recuperar o valor original: `\{\}\texttt{d[arr[i]]}`.

Time: $O(N \log N)$

```
vector<int> d = arr;
sort(d.begin(), d.end());
d.resize(unique(d.begin(), d.end()) - d.begin());
for (int i = 0; i < n; ++i) {
    arr[i] = lower_bound(d.begin(), d.end(), arr[i]) -
        d.begin();
}
```

Gen.h

Description: Template de gerador de testes aleatórios usando `\{\}\texttt{mt19937}`. Gera valores inteiros uniformemente distribuídos em um intervalo.

Time: N/A

```
#include <bits/stdc++.h>
using namespace std;
typedef long long ll;
```

```
mt19937 rng((uint32_t) chrono::steady_clock::now().time_since_
    epoch().count());
```

```
int main(){
    cin.tie(0), ios_base::sync_with_stdio(false);
    int n = uniform_int_distribution<int> (1, 3)(rng);
}
```

Makefile.h

Description: Template de Makefile com warnings e sanitizers para programação competitiva.

Usage: `\{\}\texttt{make a}`

Time: N/A

```
WARNINGS := -Wshift-overflow=2 -Wfloat-equal -Wconversion
SANITIZERS := -fsanitize=address -D_GLIBCXX_DEBUG
    -D_GLIBCXX_DEBUG_PEDANTIC -fsanitize=undefined
a.out: a.cpp
    g++ -Dlocal $(WARNINGS) $(SANITIZERS) a.cpp
```

orderedSet.h

Description: Ordered set e multiset usando GNU PBDS. Suporta `\{\}\texttt{order\{\}_of\{\}_key}` (rank) e `\{\}\texttt{find\{\}_by\{\}_order}` (k -ésimo elemento).

Time: $O(\log n)$ por operação

```
#include <ext/pb_ds/tree_policy.hpp>
#include <ext/pb_ds/assoc_container.hpp>
using namespace __gnu_pbds;
```

```
using ordered_set = tree<ll, null_type, less<ll>,
    rb_tree_tag, tree_order_statistics_node_update>;
using ordered_multiset = tree<ll, null_type, less_equal<ll>,
    rb_tree_tag, tree_order_statistics_node_update>;
```

TernarySearch.h

Description: Busca ternária para funções discretas e contínuas. Encontra o menor $i \in [a, b]$ que maximiza $f(i)$, assumindo $f(a) < \dots < f(i) \geq \dots \geq f(b)$. Para minimizar, inverter as desigualdades. Para reais, usar $\approx 200-300$ iterações.

Usage: `\{\}\texttt{int ind = ternSearch(0, n-1, [\{\}&](int i)\{\}\{ return a[i]; \{\}\});}`

Time: $O(\log(b-a))$

```
double ternary(double l, double r){
    // < for maximum and > for minimum value
    int cont = 300;
    while (cont --)
    {
        double m1 = l + (r-l)/3;
        double m2 = r - (r-l)/3;
        double f1 = f(m1);
        double f2 = f(m2);
        if(f1>f2){
            l = m1;
        }else{
            r = m2;
        }
    }
    return l;
}
```

```
/**
 * Author: Simon Lindholm
 * Date: 2015-05-12
 * License: CC0
 * Source: own work
 * Description:
 * Find the smallest i in [a,b] that maximizes f(i),
    assuming that f(a) < \dots < f(i) \ge \dots \ge f(b).
 * To reverse which of the sides allows non-strict
    inequalities, change the < marked with (A) to <=, and
    reverse
 * the loop at (B).
 * To minimize f$, change it to >, also at (B).
 * If you are dealing with real numbers, you'll need to pick
    $m_1 = (2a + b)/3.0$ and $m_2 = (a + 2b)/3.0$.
 * Consider setting a constant number of iterations for the
    search, usually [200,300]$ iterations are sufficient
 * for problems with error limit as $10^{-6}$.
 * Status: tested
 * Usage: int ind = ternSearch(0,n-1,[\{\}&](int i){return
    a[i];});
 * Time: O(\log(b-a))
 */
```

```
int ternSearch(int a, int b) {
    assert(a <= b);
    while (b - a >= 5) {
        int mid = (a + b) / 2;
        if (f(mid) < f(mid+1)) a = mid; // (A)
        else b = mid+1;
    }
    for(int i = a+1; i <= b; ++i)
        if (f(a) < f(i)) a = i; // (B)
    return a;
}
```

XorBasis.h

Description: Xor Basis. Mantém uma base de vetores e permite reduzir um valor ao mínimo XOR com elementos da base. `mx(l)` retorna o máximo valor XOR usando elementos cujo segundo campo $\geq l$.

Time: $O(\log \text{MAX})$ por operação

```
struct Basis{
    long long neutral = (1ll << 31) - 1;
    vector <pair<int, int>> basis;
```

```
    Basis(){
    }
    void add(pair<int, int> x){
        for(auto& i : basis){
            x.first = min(x.first, x.first^i.first);
        }
        if(x.first != 0){
            basis.push_back(x);
        }
    }
}
```

```
int mx(int l){
    long long x = neutral;
    for(auto& i : basis){
        if(i.second >= l){
            x = min(x, x^i.first);
        }
    }
    return x ^ neutral;
};
```